



SCHOOL OF COMPUTER SCIENCE

Bridging the Gap: Comparing Geometric Path Planning Between GPT-4o and Classical Algorithms in Indoor Navigation

Jimin Ryu

A dissertation submitted to the University of Bristol in accordance with the requirements of the degree
of Bachelor of Science in the Faculty of Engineering **worth 40CP**.

Monday 12th May, 2025

Abstract

Path planning in structured indoor environments typically based on classical geometric algorithms such as A*, which compute collision-free paths using explicit spatial representations. While these methods are precise and reliable, they depend on manual geometric specifications and lack semantic reasoning abilities. Recent advancements in Large Language Models (LLMs), particularly GPT-4o, have enhanced multimodal reasoning capabilities across text, vision, and structured data, allowing for flexible problem-solving across various modalities. However, it is still unclear how well these models handle precise spatial reasoning tasks, such as maintaining path continuity, avoiding collisions, and ensuring metric fidelity.

This project evaluates whether GPT-4o can perform geometric path planning tasks traditionally handled by classical algorithms. The experiments systematically compare GPT-4o against an A* baseline across two input types, plain 2D floorplan images, and structured Oriented Bounding Box (OBB) metadata, under zero-shot and few-shot prompting conditions.

The primary contributions of this research are:

- Implemented a complete experimental pipeline to evaluate spatial reasoning, including an A* path planner and tools for generating floorplans in both visual and structured formats.
- Conducted path planning experiments across four indoor scenes of varying complexity, testing both visual and structured inputs under zero-shot and few-shot conditions.
- Established quantitative metrics for assessing spatial reasoning in foundation models, including path validity, collision rate, and deviation from optimal paths.
- Developed an evaluation framework to systematically compare classical algorithmic approaches with LLM-based spatial reasoning across several input modalities, allowing the reasoning gap between these methods to be quantified.

This research provides a diagnostic framework for evaluating spatial reasoning in LLMs, clarifying the limits of current models and providing direction for future improvements.

Declaration

I declare that the work in this dissertation was carried out in accordance with the requirements of the University's Regulations and Code of Practice for Taught Programmes and that it has not been submitted for any other academic award. Except where indicated by specific reference in the text, this work is my own work. Work done in collaboration with, or with the assistance of others including AI methods, is indicated as such. I have identified all material in this dissertation which is not my own work through appropriate referencing and acknowledgement. Where I have quoted or otherwise incorporated material which is the work of others, I have included the source in the references. Any views expressed in the dissertation, other than referenced material, are those of the author.

Jimin Ryu, Monday 12th May, 2025

AI Declaration

I declare that any and all AI usage within the project has been recorded and noted within Appendix A or within the main body of the text itself. This includes (but is not limited to) usage of text generation methods incl. LLMs, text summarisation methods, or image generation methods.

I understand that failing to divulge use of AI within my work counts as contract cheating and can result in a zero mark for the dissertation or even requiring me to withdraw from the University.

Jimin Ryu, Monday 12th May, 2025

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Objectives	2
2	Background	4
2.1	Classical Geometric Planners for Indoor Navigation	4
2.2	End-to-End Learned Navigation Policies	5
2.3	Language Models for Navigation and Spatial Reasoning	5
2.4	Spatial Priors in Navigation	7
2.5	Hybrid Planning in Embodied Navigation	7
2.6	Semantic Mapping and Reasoning	8
3	Project Execution	10
3.1	Design Overview	10
3.2	Dataset and Scene Representation	10
3.3	Navigation Task Construction	12
3.4	A* Planner Implementation	12
3.5	Prompt Construction and GPT-4o Querying	13
4	Critical Evaluation	18
4.1	Metrics	18
4.2	Challenges in Visual Spatial Reasoning	18
4.3	Strengths of Structured Spatial Representation	20
4.4	Inconsistent Performance of Few-Shot Learning	20
4.5	Comparative Analysis with Classical A*	21
5	Conclusion	24
5.1	Contribution Summary	24
5.2	Future Work	24
A	Appendix A: AI Prompts/Tools	31
B	Flattened Floorplans with 3D Mesh and OBBs	32
C	Visual Comparison of GPT-4o and A* Path Planning	33

List of Figures

1.1	Conceptual framework for evaluating LLM spatial reasoning: transitioning from classical path planning to LLM-based path inference using image and OBB inputs, with empirical evaluation.	2
2.1	Taxonomy of classical path-planning algorithms categorised by search type (graph-based vs. sampling-based) and environment assumptions (static, dynamic, or high-dimensional).	4
2.2	Comparison between Vision-and-Language Navigation (VLN) and Visual Question Answering (VQA), adapted from [3].	5
2.3	Examples of spatial priors used in navigation: (a) topological scene graphs; (b) hierarchical metric-semantic representations, adapted from [48, 29].	7
2.4	Comparison between (a) Axis-Aligned Bounding Box (AABB) and (b) Oriented Bounding Box (OBB), adapted from [27].	8
2.5	Comparison and integration of classical, learned, and hybrid planning approaches in embodied navigation.	9
3.1	Sorted floorplans used in experiments. The images are arranged from the most complex and cluttered (top left) to the simplest and sparsest (bottom right). Each floorplan is rendered from the ASE dataset, illustrating the object footprints and their semantic labels. The complexity is determined based on area size, object density, and spatial arrangement. . . .	15
3.2	Example snippet of structured OBB metadata in JSON format	16
3.3	Floorplan-based Prompt Structure for GPT-4o	16
3.4	Metadata-based Prompt Structure for GPT-4o	17
4.1	Zero-Shot Learning Error Rates for Floorplan-Based Navigation	19
4.2	Few-Shot Learning Error Rates for Floorplan-Based Navigation	19
4.3	Comparison of Incompletion Rates: Few-Shot vs. Zero-Shot Learning across Scenes. . . .	20
4.4	Comparison of Path Validity and Collision Rates between Zero-Shot and Few-Shot Learning across Scenes.	21
4.5	Comparison of Success Rates (Path Validity) Across Scenes between Few-Shot and Zero-Shot Learning.	21
4.6	Comparison of Global Path Length Ratios between Few-Shot and Zero-Shot Learning: Valid vs. Colliding Paths.	22
4.7	Comparison of Global Waypoint Deviations between Few-Shot and Zero-Shot Learning: Valid vs. Colliding Paths.	22
C.1	A* Path for Scene 81056	34
C.2	Zero-Shot Path for Scene 81056	34
C.3	Few-Shot Path for Scene 81056	34
C.4	Comparison of A*, Zero-Shot, and Few-Shot Paths for Scene 81056	34
C.5	A* Path for Scene 84230	35
C.6	Zero-Shot Path for Scene 84230	35
C.7	Few-Shot Path for Scene 84230	35
C.8	Comparison of A*, Zero-Shot, and Few-Shot Paths for Scene 84230	35
C.9	A* Path for Scene 84052	36
C.10	Zero-Shot Path for Scene 84052	36
C.11	Few-Shot Path for Scene 84052	36
C.12	Comparison of A*, Zero-Shot, and Few-Shot Paths for Scene 84052	36

C.13 A* Path for Scene 83692	37
C.14 Zero-Shot Path for Scene 83692	37
C.15 Few-Shot Path for Scene 83692	37
C.16 Comparison of A*, Zero-Shot, and Few-Shot Paths for Scene 83692	37

List of Tables

3.1	Number of Navigation Tasks per Scene	12
3.2	In-Context Example Scenes for Floorplan-Based Few-Shot Condition.	14

Ethics Statement

This project did not require ethical review, as determined by my supervisor, Shawn Shen.

Supporting Technologies

- I used the OpenAI API, accessed via the OpenAI Python client (1.75.0), to integrate GPT-4o into the navigation pipeline.
- I used Python (3.10.12) as the main programming environment to develop navigation pipelines, floorplan generation tools, and path planning systems.
- I used Matplotlib (3.9.0) to visualise floorplans, object footprints, and generated paths.
- I used NumPy (1.26.4) for numerical operations, depth processing, pose transformations, and coordinate handling.
- I used PyTorch (2.3.0+cu121) to process camera calibration tensors, perform coordinate transformations and 2D projection operations during floorplan and OBB generation.
- I used Trimesh (4.4.9) to slice 3D meshes, generate Oriented Bounding Boxes (OBBs), and process mesh geometry.
- I used Shapely (2.1.0) to perform geometric operations such as collision detection between paths and object footprints.
- I used the `pyquaternion` library to handle quaternion-based rotations when computing OBB transformations from 3D ground truth meshes.
- I used the `efm3d` library to load camera data, snippet streams, and calibration parameters from Project Aria sequences.
- I used the Aria Synthetic Environments (ASE) dataset from Project Aria (<https://www.projectaria.com/datasets/ase/>) to obtain 3D indoor scene meshes and 3D OBBs for generating floorplans.

Notation and Acronyms

OBB : Oriented Bounding Box
LLM : Large Language Model
VLN : Vision-and-Language Navigation
IQR : Interquartile Range

Chapter 1

Introduction

1.1 Motivation

Path planning in structured environments has traditionally relied on classical geometric algorithms such as A*, D*, and Rapidly-Exploring Random Trees (RRT) [28, 51, 33]. These planners operate over explicit spatial representations, including occupancy grids, or metric maps, and compute collision-free trajectories using deterministic search heuristics. Their reliability and optimality in well-mapped environments have established them as the standard for autonomous indoor navigation systems [42, 37]. Despite their strengths, these methods remain constrained. They require manually specified geometry, function at low-level coordinate spaces, and lack the capacity to reason about semantics, object affordances, or high-level task context.

Recent advances in large-scale pre-training have driven the development of Large Language Models (LLMs) such as GPT-4o, which show strong performance across tasks involving language, vision, and symbolic reasoning [40, 49]. A unified interface presented by these models enable interacting with diverse modalities and problem structures without retraining across domains. In particular, GPT-4o’s ability to process images, text, and structured data provides a solid foundation for adaptable, general-purpose agents capable of goal-directed reasoning in complex environments.

Recent work has applied LLMs to high-level navigation tasks [13]. These often involve natural language instructions through semantic scene understanding [2, 44, 3]. However, most of these systems rely on models integrated into end-to-end architectures that are fine-tuned for specific environments. In contrast, directly using LLMs to interpret spatial relationships from visual or structured inputs, without explicit training for navigation, remains largely unexplored. While LLMs have shown success in semantic reasoning, their ability to perform precise spatial reasoning, including path continuity, collision avoidance, and metric fidelity, is still not well understood.

This work addresses that gap. Instead of designing a novel navigation system, we adopt an evaluative approach by empirically examining whether GPT-4o can perform low-level path planning tasks that are traditionally handled by classical algorithms. We consider the language model not as a semantic controller but as a geometric reasoner, responsible for producing collision-free paths from a start to a goal location, based on two distinct input formats:

- Plain 2D floorplan images.
- Structured Oriented Bounding Box (OBB) metadata.

We systematically evaluate GPT-4o under multiple prompting paradigms:

- Zero-shot: the model receives no prior examples.
- Few-shot: the model is provided with 2 correct in-prompt demonstrations.

These experiments aim to quantify the spatial reasoning capabilities of GPT-4o and to identify the boundary between what current foundation models can infer geometrically, and what still requires explicit algorithmic planning. We benchmark all LLM outputs against an A* planner operating on ground-truth OBB maps, which generates a collision-free reference trajectory.

This evaluation is motivated by several key research questions:

1. Can LLMs generalise to geometric planning tasks with no prior training?

2. Does few-shot prompting significantly improve the model’s spatial reasoning?
3. What is the impact of input representation (visual vs. structured) on planning accuracy?
4. Where do LLM-based planners fail, and why?

By addressing these questions, this project introduces a novel diagnostic framework for evaluating the geometric reasoning capabilities of large-scale multimodal models. In contrast to existing research that focuses semantic planning or policy learning for task execution, our focus is explicitly on spatial inference, using path planning as a structured task to assess the reasoning abilities of language models. The objective is to investigate how well language models can approximate or acquire geometric priors implicitly without explicit supervision or task-specific training, rather than to replace classical planning algorithms.

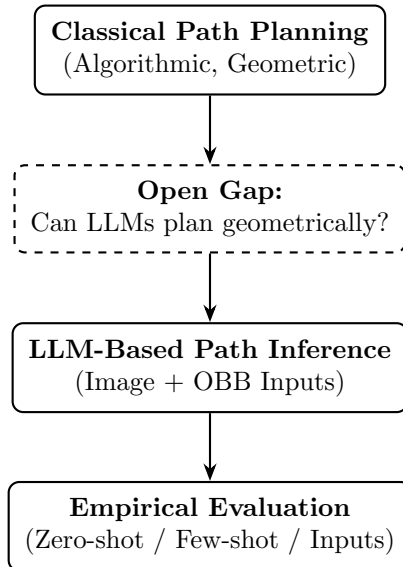


Figure 1.1: Conceptual framework for evaluating LLM spatial reasoning: transitioning from classical path planning to LLM-based path inference using image and OBB inputs, with empirical evaluation.

This project approaches path planning not as a solved technical challenge, but as a diagnostic framework for investigating the implicit geometric understanding encoded within modern LLMs. By systematically comparing their performance with classical algorithms, we aim to establish an empirical baseline for spatial reasoning in foundation models, highlighting both their current limitations and their potential for future development.

1.2 Objectives

This project aims to empirically investigate the capabilities and limitations of LLMs, specifically GPT-4o, in contrast to classical geometric methods for indoor navigation. The primary objective is to evaluate how well GPT-4o can approximate geometric path planning, a task traditionally addressed by algorithms such as A*, when given visual or structured inputs. The project assesses spatial reasoning accuracy, identifies common failure cases, and analyses the model’s sensitivity to different input representations across multiple prompting paradigms.

The objectives of this dissertation are as follows:

1. **Establish a classical geometric baseline** by implementing A* path planning on ground-truth 2D Oriented Bounding Box (OBB) footprint maps. This provides a deterministic benchmark with guaranteed collision-free trajectories.
2. **Evaluate GPT-4o’s coordinate-based planning from visual input**, using plain 2D floorplan images. We assess its ability to infer spatial structure and generate valid waypoint sequences, and identify common failure cases such as collision, disconnection, or goal misspecification.

3. **Evaluate coordinate-based planning from structured OBB metadata**, testing GPT-4o’s spatial reasoning with structured, geometry-aware input. We examine whether providing explicit object boundaries improves accuracy and reduces failure rates compared to image-based input.
4. **Systematically compare LLM-driven planning with the A* baseline** across all conditions. We quantify performance differences, analyse the effects of input modalities and prompting paradigms, and identify specific limitations in GPT-4o’s geometric understanding to inform future multimodal navigation research.

These objectives are intended to provide a systematic and modality-aware evaluation of spatial reasoning in LLMs. The findings can guide future research and development in improving spatial reasoning capabilities and integration of LLMs in navigation tasks.

Chapter 2

Background

2.1 Classical Geometric Planners for Indoor Navigation

Classical path-planning algorithms such as A* [28], D* [51], and Rapidly-exploring Random Trees (RRT) [33] form the foundation of autonomous navigation in structured environments [42, 37, 30]. These methods operate on discrete or continuous geometric maps, such as grids or graphs [28, 51, 33, 18], and utilise heuristics to compute shortest or safest paths [28, 51, 18]. In particular, A* guarantees optimal performance on a graph with admissible heuristics [28], and has been widely used due to its simplicity and effectiveness [19]. A* selects the node n that minimises the estimated total cost at each step:

$$f(n) = g(n) + h(n) \quad (2.1)$$

where $g(n)$ is the known cost from the start node to n , and $h(n)$ is the heuristic estimated cost from the current node n to the goal. A* guarantees an optimal path, given that $h(n)$ is admissible, meaning it never overestimates the true cost [28].

Classical planning methods can be categorised based on search strategy and environment assumptions, as illustrated in Figure 2.1.

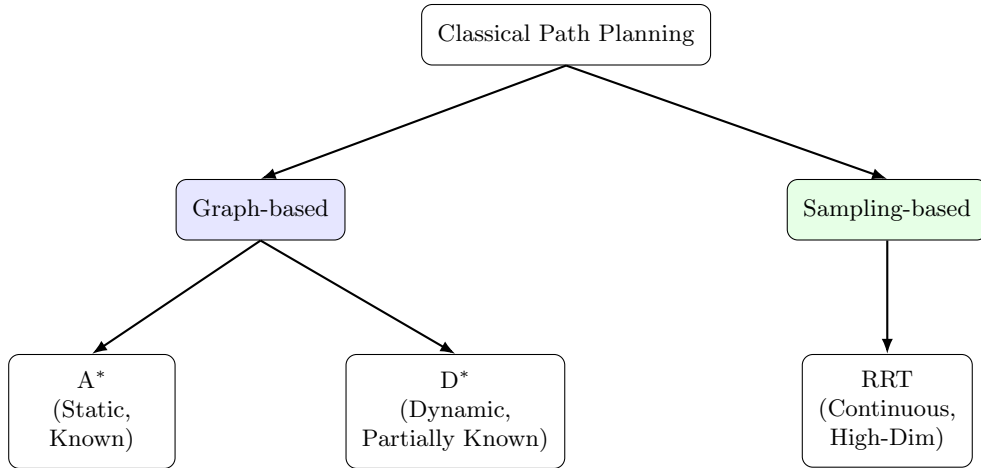


Figure 2.1: Taxonomy of classical path-planning algorithms categorised by search type (graph-based vs. sampling-based) and environment assumptions (static, dynamic, or high-dimensional).

In indoor navigation, planners are often extended with collision-aware cost functions that penalise proximity to obstacles or narrow passages [62]. These modifications improve real-world practicality, but require dense and accurate environmental models, such as 2D maps or 3D reconstruction with known obstacles [6, 60]. Classical planners therefore remain a strong baseline for structured indoor navigation, particularly when paired with accurate spatial representations. However, their effectiveness is constrained by their reliance on low-level geometric information, which lacks the ability to interpret semantic context or adapt to high-level task goals. Although these methods can compute solid collision-free paths, they are fundamentally limited in their capacity to reason about object semantics, functional relevance, or goal-

oriented purpose. These limitations become more apparent in tasks that require contextual understanding or interaction beyond geometric path planning.

In this work, A* search is used as the geometric baseline, utilising ground-truth Oriented Bounding Box (OBB) footprints as input. This provides a strong classical baseline for comparison with language-model-based planning.

2.2 End-to-End Learned Navigation Policies

Over the past decade, learning-based navigation methods have emerged as alternatives to classical planners, aiming to infer navigation behaviours directly from perceptual inputs such as RGB-D images, rather than relying on geometric maps or handcrafted heuristics. These approaches were enabled by large-scale datasets such as Matterport3D [7], and the development of photorealistic 3D simulators such as Habitat [47], which allow agents to learn through interaction in visually rich, goal-directed environments.

Vision-and-Language Navigation (VLN) tasks [3] extend this idea by introducing natural-language instructions, requiring agents to ground linguistic commands in visual scenes. Datasets such as Room-to-Room (R2R) and environments such as the Matterport3D Simulator support this line of research, positioning navigation as a sequential decision-making problem that integrates perception, language, and control. Figure 2.2 illustrates the contrast between VLN and static vision-language tasks like Visual Question Answering (VQA), showing that VLN requires dynamic viewpoint manipulation over time.

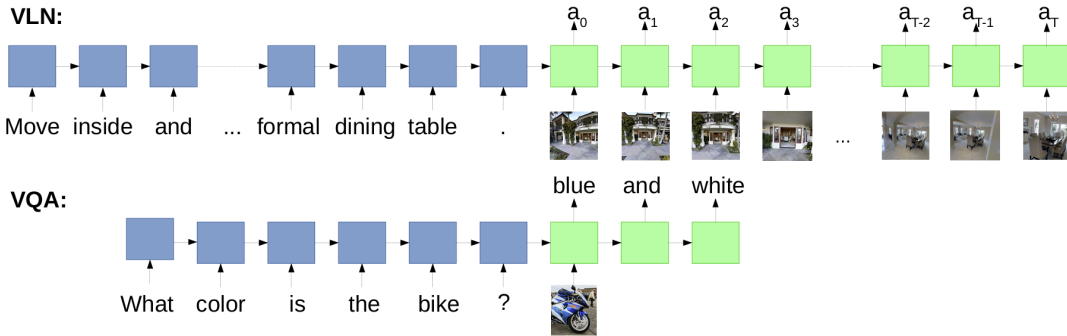


Figure 2.2: Comparison between Vision-and-Language Navigation (VLN) and Visual Question Answering (VQA), adapted from [3].

Learned navigation policies span a range of techniques, from supervised learning of instruction-following behaviour [20], to reinforcement learning in large-scale indoor environments [8, 9], to transformer-based agents that incorporate longer temporal and spatial context [11]. These methods have demonstrated strong generalisation to new layouts without requiring explicit geometric maps. However, they typically require extensive training, environment-specific fine-tuning, and large amounts of annotated data [52, 21], limiting their applicability in low-resource or task-specific settings.

In contrast to these learned policies, our approach uses pre-trained large language models, specifically GPT-4o, to perform spatial reasoning directly. Given that our dataset is small and static, we chose to avoid end-to-end training. This approach reframes navigation as an evaluation of general-purpose spatial reasoning ability, rather than deriving navigation strategies through training.

2.3 Language Models for Navigation and Spatial Reasoning

Large Language Models (LLMs) have demonstrated strong reasoning capabilities across textual domains, enabling structured decision-making and multi-step problem solving. Techniques such as step-by-step intermediate computation [39] and chain-of-thought prompting [57] allow models to decompose complex tasks into interpretable reasoning steps. These capabilities now support the application of LLMs beyond simple text generation, allowing them to perform logical inference, answer complex questions, and carry out symbolic manipulation.

Recent advances have extended these reasoning abilities into the multimodal domain [61]. Multimodal Large Language Models (MLLMs) enable agents to reason over images, language, and structured data by jointly processing visual, linguistic, and symbolic inputs within a unified framework. For instance,

CLIP [43] learns aligned visual-text embeddings from large-scale paired data, enabling open-vocabulary recognition and zero-shot classification. Following this approach, PaLM-E [16] integrates perceptual embeddings directly into a language model, allowing visual-linguistic reasoning across various embodied tasks. This multimodal fusion allows agents to interpret and navigate complex indoor scenes not only through geometric mapping, but also through high-level semantic cues and contextual understanding.

Building on these foundations, recent systems have expanded the role of language models from passive scene interpretation toward goal-directed spatial reasoning and semantic-level planning. SayPlan demonstrates this shift by using LLMs to query hierarchical scene graphs and generate task-relevant plans in large-scale 3D environments [44]. Similarly, SayCan combines semantic reasoning from LLMs with learned skill value functions to enable mobile robots to follow abstract, long-horizon instructions in real-world scenarios [2].

These systems show a broader transition in embodied AI from purely geometric planning toward hybrid reasoning pipelines that integrate spatial structure with semantic understanding. Recent models such as GPT-4o [40] unify visual processing, language-based reasoning, and structured inputs within a single, general-purpose framework. This suggests that such models could function as flexible agents for goal-directed tasks, capable of reasoning across perception, geometry, and semantics without requiring task-specific training.

2.3.1 Zero-Shot, Few-Shot, and Fine-Tuning Paradigms

LLMs are applied through distinct learning paradigms depending on how task-specific knowledge is incorporated at inference or training. The main paradigms are zero shot, few shot, and fine tuning. These approaches define how much prior exposure the model has to the specific task and a different method of incorporating that knowledge, whether through in-context examples, additional supervised training, or no prior information [56, 32].

Zero-Shot Learning In the zero shot setting, LLM is expected to perform a task without any task-specific training examples or demonstrations [59, 10]. The model is only given the task input and a textual instruction, without any additional guidance. This setting is used to evaluate the model’s ability to generalise from pre-training alone, without task-specific supervision. This approach is commonly used to evaluate generalisation in both textual and multimodal tasks, especially in low-resource or open-ended domains with limited or no labelled data. However, it often underperforms on tasks which need precise formatting, detailed instructions, or domain specific knowledge.

Few-Shot Learning In the few shot setting, task-specific examples are embedded directly into the prompt to guide the model’s behaviour without updating its parameters. This form of task conditioning occurs entirely through prompting rather than gradient-based learning. Few shot prompting utilises the model’s ability to identify patterns, analogies, and formats from minimal supervision. The release of GPT-3 showed that few shot prompting can be an effective method, achieving strong performance on natural language processing tasks such as translation, answer generation, and reasoning, without any additional training [5]. This method helps to avoid full training by using a small number of in-prompt demonstrations, but its effectiveness depends on the prompt design and the quality of the examples.

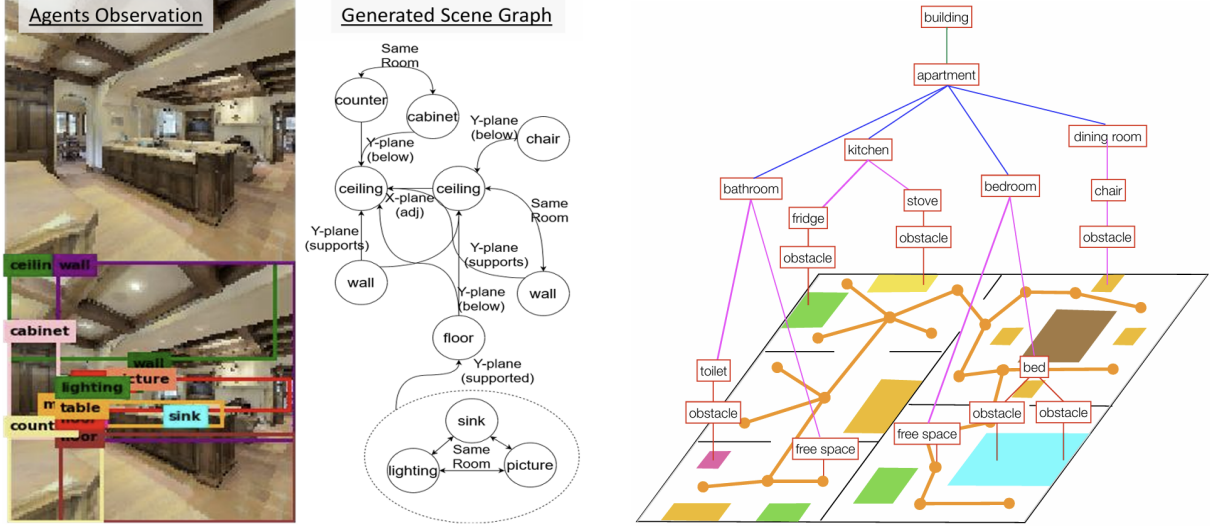
Fine-Tuning Fine tuning trains a pre-trained model further on a task specific dataset using gradient descent. Unlike zero-shot and few-shot approaches, it updates the model’s weights to improve performance on a specific task or domain. This method often gives higher accuracy when large amounts of well-labelled data and computational resources are available, especially for specialised applications. Fine tuned models often perform better than zero-shot and few-shot baselines on benchmarks where the training data aligns closely with the evaluation domain [26]. However, it risks overfitting and unintended biases.

These paradigms are not mutually exclusive and are often combined in practice. For instance, a fine tuned model may still benefit from few shot prompting to adjust its output style or reasoning strategy [12]. Recent research positions in context learning as a strong alternative to full fine tuning, particularly for tasks involving structured reasoning [57], multimodal inputs, or limited annotated data [17, 54].

In this project, we specifically focus on evaluating GPT-4o’s spatial reasoning abilities using zero-shot and few-shot settings, without any gradient-based fine-tuning. This allows us to investigate the model’s spatial reasoning capabilities without relying on task-specific training. This approach aligns with recent research that explores LLMs as general purpose planners.

2.4 Spatial Priors in Navigation

Spatial priors provide structural information about an environment that agents can utilise for planning and decision-making. These priors are commonly represented in the form of metric maps, occupancy grids, semantic annotations, or structured geometric representations such as OBBs. In classical planning systems, spatial priors are typically assumed to be accurate and complete, enabling efficient path computation using geometric heuristics [28, 51, 18]. In contrast, learning-based approaches encode prior knowledge through topological graphs [48], spatial memory modules [34], or hierarchical map representations [29], as illustrated in Figure 2.3.



(a) Scene graph generated by the method in [48]. It includes the agent’s observation, detected object nodes with bounding boxes, and the resulting relational graph.

(b) Hierarchical metric-semantic map with compressed sub-symbolic layers from [29], representing free space and objects via bounding polygons and sparse topological nodes.

Figure 2.3: Examples of spatial priors used in navigation: (a) topological scene graphs; (b) hierarchical metric-semantic representations, adapted from [48, 29].

OBBs are widely used in 3D object detection [41] and spatial reasoning [50] to represent object footprints and orientations with low overhead. OBBs can rotate to match the local orientation of objects, unlike Axis-Aligned Bounding Boxes (AABBs), which are restricted to alignment with the global coordinate axes [25]. As shown in Figure 2.4, this allows OBBs to produce tighter and more accurate spatial representations. This distinction is particularly useful in cluttered or structured indoor environments, where AABBs may overestimate the occupied space and lead to excessively conservative path planning. OBBs support more effective high-level reasoning by offering more precise spatial abstraction and reducing sensitivity to visual noise.

In our experiments, we use accurately annotated 3D OBBs as spatial priors. These are projected into 2D top-down floorplans to enable visual planning based on spatial structure, preserving geometric accuracy while simplifying spatial representation. By using ground-truth data, we isolate the spatial reasoning performance of GPT-4o from noise in spatial inputs. We do not evaluate the OBB detection process itself, as our primary focus is on how GPT-4o interprets and plans paths using accurate spatial representations. Future work could incorporate learning-based OBB fitting from raw scans to evaluate robustness in real-world conditions.

2.5 Hybrid Planning in Embodied Navigation

Hybrid planning represents the integration of multiple reasoning modalities, such as geometric and semantic, within a single navigation system [36]. Classical planning methods which rely on metric maps and cost-based heuristics provide accurate spatial reasoning and collision avoidance, but lack flexibility for high-level task goals or semantic context, as discussed in Section 2.1. In contrast, learning-based systems based on language and vision offer more adaptive decision-making, but with limited generalisation and lack spatial guarantees, as outlined in Section 2.2.

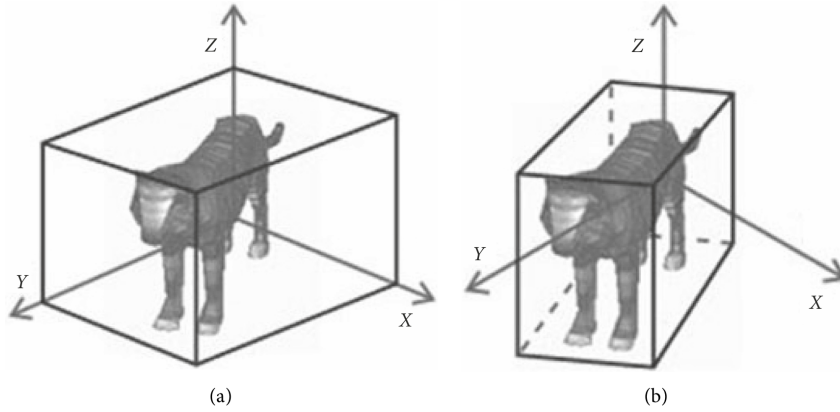


Figure 2.4: Comparison between (a) Axis-Aligned Bounding Box (AABB) and (b) Oriented Bounding Box (OBB), adapted from [27].

Hybrid systems aim to combine the strengths of both classical and learned approaches by embedding semantic knowledge into classical planning architectures [13]. For instance, classical planners such as A* or Transition-based RRT (T-RRT) can be augmented with semantic layers that encode traversability or visibility constraints, enabling agents to reason over environmental context during the planning process [1].

Other recent approaches use pre-trained or learned semantic models to guide symbolic or geometric navigation. SEEK incorporates object relevance through a Relational Semantic Network (RSN) derived from large language models, estimating the likelihood of target object locations and focusing the search on the promising areas [24]. Multi-Object Navigation (Multi-ON) separates symbolic waypoint planning from high-level semantic inference, combining classical Simultaneous Localisation and Mapping (SLAM) with neural modules trained to interpret visual cues and goal relevance [46]. Similarly, ReVoLT uses structured relational graphs for semantic reasoning and couples this with an intermediate Voronoi-based planner to generate context-aware subgoals [35]. These modular architectures show that semantic understanding actively influences spatial decisions across multiple levels of the navigation pipeline.

This transition toward modular hybrid frameworks has been widely explored in recent years, especially with the integration of pre-trained language models as semantic evaluators or decision modules [53]. These architectures provide improved interpretability and flexibility, while retaining geometric consistency. As a result, hybrid planning is regarded not just as a compromise, but as a unifying principle for embodied agents operating in visually complex, real-world environments.

While hybrid planning is effective for achieving more human-like navigation, understanding the individual capabilities of components like GPT-4o is essential for integration. Therefore, we evaluate the spatial reasoning capabilities of GPT-4o to determine whether it can independently perform geometric path planning from static visual or structured inputs. Understanding GPT-4o’s standalone abilities will help integrate these capabilities effectively into hybrid systems in future work.

2.6 Semantic Mapping and Reasoning

Semantic representations capture higher-level conceptual meaning such as object categories, spatial relationships, functional affordances, and task-specific relevance that purely geometric maps cannot represent or encode [45, 31]. This semantic abstraction allows agents to reason not only about where to navigate, but also why certain locations or paths are contextually important within a given task.

Early semantic navigation methods used handcrafted object models and fixed vocabularies to link spatial maps with symbolic labels [22, 55]. Semantic SLAM extended this by associating class probabilities with spatial landmarks, integrating metric and semantic information for localisation [4]. However, these systems rely on predefined categories and do not adapt dynamically to open-ended tasks.

More recent work with MLLMs moves beyond static labels toward context-aware reasoning signals. MLLMs interpret visual inputs and linguistic prompts jointly, generating flexible inferences, such as evaluating route navigability, identifying goal-relevant objects, or inferring unsafe regions, rather than relying on pre-annotated symbolic maps [58, 15].

Semantic information thus evolves from an auxiliary annotation into an active planning input, en-

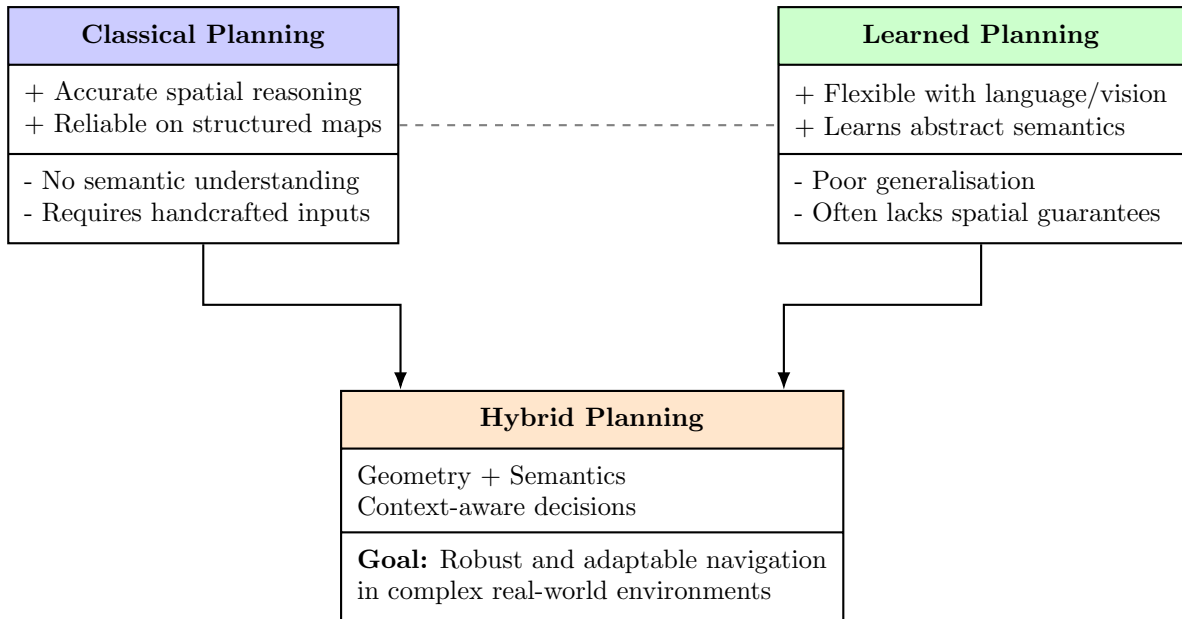


Figure 2.5: Comparison and integration of classical, learned, and hybrid planning approaches in embodied navigation.

abling adaptive, goal-oriented strategies through context-aware decision-making across visual and linguistic modalities.

This project aims to systematically evaluate the fundamental spatial reasoning abilities of GPT-4o without contextual cues before integrating high-level reasoning. This approach will help determine how and to what extent semantic reasoning can enhance the model’s capabilities, leading to more effective use of its semantic potential in future work.

Chapter 3

Project Execution

This chapter outlines the design, methodology, and implementation of the experiments developed to evaluate the spatial reasoning capabilities of GPT-4o in indoor navigation, computing collision-free paths between a designated start and goal object in 2D scenes. Each condition is benchmarked against A* planning using ground-truth Oriented Bounding Box (OBB) representations to establish a reliable geometric baseline.

3.1 Design Overview

The project investigates whether GPT-4o, a large multimodal language model, can perform low-level geometric reasoning tasks such as coordinate path planning without any fine-tuning. To isolate this capacity, we implemented a set of controlled experiments across different modalities and prompting paradigms:

- **A* Planning (Baseline):** Deterministic path planning on 2D OBB maps using A* search.
- **GPT-4o Zero-Shot:** Coordinate-based path planning from (a) 2D floorplan images and (b) structured OBB metadata, using only natural language instructions without in-context examples.
- **GPT-4o Few-Shot:** The same tasks as above, but with two in-context A* examples included in the prompt.

Each experimental condition was designed to test GPT-4o’s spatial reasoning with different input types (image vs. structured) and increasing supervision (zero-shot vs. few-shot), without any gradient-based fine-tuning or task-specific training.

3.2 Dataset and Scene Representation

We selected a subset of procedurally generated indoor scenes from the Aria Synthetic Environments (ASE) dataset [38], a fully simulated environment dataset designed for scalable 3D scene understanding. Unlike real-world 3D scans, ASE avoids privacy concerns and provides complete, noise-free ground-truth annotations. We selected four indoor scenes with diverse spatial layouts and varying levels of clutter, while excluding those containing missing, overlapping, or excessively clustered OBBs. This allows controlled evaluation of spatial reasoning in both structured and cluttered environments.

We utilised the 3D scene mesh and its corresponding OBB annotation file, which includes full 6-DoF (Degrees of Freedom) object pose information (translation and rotation), object dimensions (width, height, depth), semantic labels, and instance IDs. These annotations are used to construct two types of spatial input representations for GPT-4o: (1) rendered 2D floorplan images, and (2) structured OBB metadata.

3.2.1 2D Floorplan Generation from 3D Meshes and Object-Level Annotations

To convert 3D scene data into top-down 2D floorplan images, we implemented a multi-step pipeline comprising data preprocessing and floorplan generation.

Data Preprocessing

For each scene, we generated a top-down 2D floorplan image by rendering a combined 3D mesh of the scene and its OBBs, projecting their geometry by ignoring the Z-axis to visualise their 2D spatial layout. The process involves the following steps:

1. **Mesh loading and OBB fusion:** The 3D scene mesh is loaded from the EFM3D EVAL Meshes, and ground-truth OBBs are parsed from CSV. We rendered the OBBs as 3D wireframe meshes and overlaid them on the scene geometry.
2. **Wall detection from mesh slicing:** We extracted a horizontal slice from the 3D mesh near the top of the scene (e.g., at 99% of the mesh height) to identify wall boundaries. These were then converted into long, thin 3D OBBs representing structural barriers.

2D Floorplan Generation

Once the combined 3D mesh is prepared, we generated a 2D floorplan image as follows:

1. **Footprint projection:** We projected all OBBs, including both ground truth and detected walls, onto a 2D plane by discarding their vertical (Z-axis) component to produce in a spatial footprint for each object.
2. **Rendering and labelling:** We rendered the 2D footprints of objects in different colours according to their semantic class. Abbreviated text labels were placed at each object’s centroid to indicate its class and instance ID.
3. **Image formatting:** To ensure consistent interpretation by GPT-4o, we adjusted the layout of the floorplan to a square aspect ratio. We also set a fixed DPI to maintain image quality and uniform scaling.

We used this generated floorplan image as input to GPT-4o in the image-based planning condition. It provides spatial layout information without precise coordinate data, requiring the model to perform visual grounding to identify the start and goal objects and compute a valid navigation path.

Generated Floorplans for Experiments

The Figure 3.1 shows the four floorplans selected for the experimental evaluation of GPT-4o’s spatial reasoning. These indoor scenes have different layout complexity and object density to assess path-planning capabilities under various conditions.

Originally, the 2D floorplans were generated by flattening the complete 3D mesh along with the projected OBBs, as shown in Appendix B. However, to ensure experimental consistency and optimise the input format for GPT-4o, we chose to use floorplans generated solely from OBBs. This approach reduces visual noise from non-essential 3D geometry, resulting in a clearer representation of spatial relationships between objects and making it more interpretable for language model-based reasoning.

3.2.2 Structured OBB Metadata Representation

In addition to floorplan images, we generated structured inputs that include precise geometric and semantic structure from the projected 3D OBBs, as illustrated in Figure 3.2. Each scene-level metadata file contains:

- **floorplan_dimensions:** Spatial extent of the 2D scene, including the minimum and maximum x and y coordinates, as well as the total width and height.
- **obb_metadata:** A list of object-level entries, each containing:
 - **footprint:** A list of 2D (x, y) points defining the object’s footprint polygon
 - **center_2d:** 2D coordinates of the OBB’s centroid after vertical projection
 - **sem_id:** Numerical semantic class ID
 - **name:** Semantic category label (e.g., "bed", "sofa")
 - **instance:** Unique instance ID for distinguishing repeated objects

- **walkable, bypassable**: Binary flags indicating whether the object is navigable or can be bypassed

We used this structured format as input to GPT-4o in the metadata-based planning condition. Unlike the image input, it provides the model with direct access to object positions and orientations, removing the need for visual inference.

3.3 Navigation Task Construction

Navigation tasks are defined as object-to-object path planning problems within each scene. Each task specifies a start and goal object by their semantic category and instance ID. Object pairs were manually selected to guarantee uniqueness and geometric feasibility. Start and goal objects were chosen to cover diverse spatial configurations, including nearby and distant pairs, and varying levels of clutter between endpoints. To maintain consistent evaluation across input modalities, we used the same set of tasks for both the image-based and metadata-based navigation conditions. The scenes used for these navigation tasks are illustrated in Figure 3.1, which provides visual context for the layout and object density of each floorplan.

Table 3.1: Number of navigation tasks (start-goal pairs) defined for each experimental scene. The pairs are chosen based on the distance between object and the level of clutter around each object.

Scene ID	Number of Navigation Tasks
81056 (Very large & cluttered)	12 pairs
84230 (Large & moderately dense)	11 pairs
84052 (Medium size & density)	9 pairs
83692 (Small & sparse)	7 pairs

3.4 A* Planner Implementation

We implemented the A* planner as a baseline to perform efficient path planning within indoor environments, specifically for evaluating the spatial reasoning capabilities of GPT-4o in navigation tasks. The planner computes collision-free paths between a designated start and goal object within 2D scenes. The following outlines the implementation details of the A* planner.

3.4.1 Initialisation

The A* planner class is initialised with the following parameters:

- **obb_list**: A list of OBBs representing obstacles and navigable objects.
- **floorplan_dims**: Boundaries of the 2D scene (`min_x`, `min_y`, `max_x`, `max_y`).
- **grid_size**: The resolution of the grid in meters (default: 0.1 m).

At initialisation, the planner computes the grid dimensions and initialises a cost map as a 2D array, with each cell holding the traversal cost. The obstacle data from OBBs is converted into polygonal representations for spatial analysis, allowing the planner to incorporate both structural and semantic costs.

3.4.2 Cost Map Initialisation

The cost map is constructed through the following steps:

1. **Obstacle Marking**: OBB footprints are converted to polygons. High costs are assigned to walls (`sem_id = 99`) to indicate impassable regions. Additionally, floor mats (`sem_id = 5`) are given a moderate traversal cost of 3, reflecting a more navigable surface. All other obstacles are assigned a default traversal cost of 50, representing moderate difficulty.

2. **Contextual Cost Map Update:** The cost map is updated at the beginning of each pathfinding task to ignore the source and target objects. This ensures that paths are computed without being blocked by the starting or ending points.
3. **Grid Update:** Each grid cell is checked to determine if the point it represents lies within any obstacle polygon. If it does, the cost map is updated based on the polygon’s semantic classification.
4. **Distance Transform and Bottleneck Detection:** An obstacle mask is created to calculate the Euclidean distance to the nearest obstacle. The planner also detects narrow passageways (bottle-necks) and applies a penalty to paths that pass through these constrained areas.

3.4.3 Pathfinding using A* Search

The pathfinding method utilises the A* algorithm to find the optimal path between start and goal coordinates:

1. The heuristic function calculates the Euclidean distance between the current node and the goal.
2. The open set is managed using a priority queue to ensure nodes with the lowest estimated cost are expanded first.
3. The algorithm iterates through neighbouring nodes (8-directional movement) to explore possible paths while avoiding obstacles.
4. If a path is found, collision checking is performed on the final path to identify intersections with high-cost polygons. The collision detection process involves creating line segments between consecutive waypoints and checking each segment for intersections with obstacle polygons.
5. The path length, collision data, and the waypoints are saved as a JSON file for post-experiment evaluation.

We used the A* planner as a geometric baseline to systematically evaluate GPT-4o’s spatial reasoning abilities. By providing a consistent and deterministic method for indoor path planning, it allows for a quantitative comparison between classical algorithms and the multimodal reasoning approaches employed by GPT-4o.

3.5 Prompt Construction and GPT-4o Querying

To evaluate GPT-4o’s spatial reasoning, prompts were constructed for both input modalities:

- **Visual input:** The 2D floorplan image is embedded as a base64-encoded data URL and passed to the model within a structured message format. The message includes a textual instruction (e.g., “Compute path from ‘BE.19’ to ‘SO.28’.”). The complete prompt is shown in Figure 3.1.
- **Structured input:** The structured metadata input consists of a JSON file containing OBB data, as shown in Figure 3.2. The prompt message is constructed by specifying the navigation task using object names and instances with their corresponding coordinates. The essential structure is shown in Figure 3.2.

In both conditions, GPT-4o is prompted to find a collision-free path.

3.5.1 Zero-Shot

In the zero-shot condition, only the current scene input and the navigation instruction are provided, without any additional examples or formatting guidance.

3.5.2 Few-Shot

In the few-shot learning condition, the prompt message is appended with two in-context examples consisting of object pairs, a scene image, and A* generated waypoints.

We used different few-shot prompting strategies for floorplan-based and metadata-based inputs to account for their distinct spatial characteristics and input structures.

Few-Shot Example Selection for Floorplan-Based Input

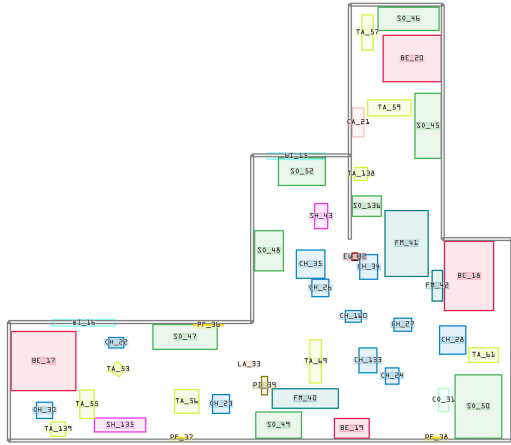
Table 3.2 lists the in-context example scenes selected for each test scene in the floorplan-based input condition. For each example scene, one pair is chosen, making a total of two pairs used as examples. The criteria for selecting scenes are size, density, and complexity to provide GPT-4o with both similar and contrasting examples. The corresponding floorplans, illustrating layout complexity and object density, are shown in Figure 3.1.

Table 3.2: In-Context Example Scenes for Floorplan-Based Few-Shot Condition. The table outlines the chosen example scenes with the selection reason based on size, density, and complexity.

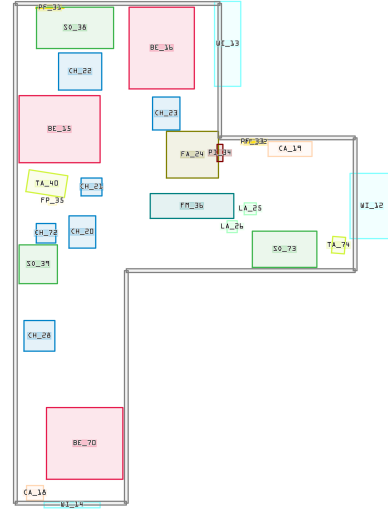
Test Scene	In-Context Example Scenes	Selection Reason
81056	84230, 84052	84230: Next largest with similar density. 84052: Tall and medium complex, helps handle large and cluttered layouts.
84230	81056, 84052	81056: Larger and more complex. 84052: Similar height and medium density, providing diverse complexity.
84052	83692, 84230	83692: Smallest and simplest, providing a less cluttered example. 84230: Larger and more complex, providing contrast.
83692	84052, 84230	84052: Next smallest with medium complexity. 84230: Large and cluttered, enhancing diversity.

Few-Shot Example Selection for Metadata-Based Input

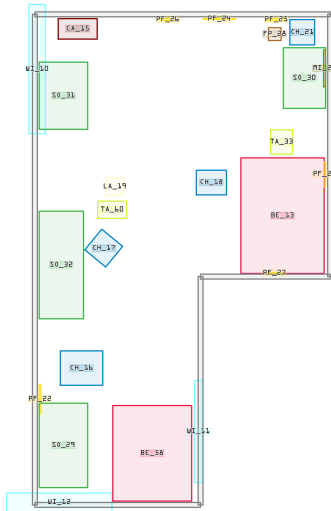
For the metadata-based input, we used two in-context examples from the same scene that differ in length and complexity, consisting of start and goal pairs not included in the testing set. This approach allows the model to learn from various navigation tasks while maintaining scene consistency, enabling us to evaluate in-context geometric reasoning.



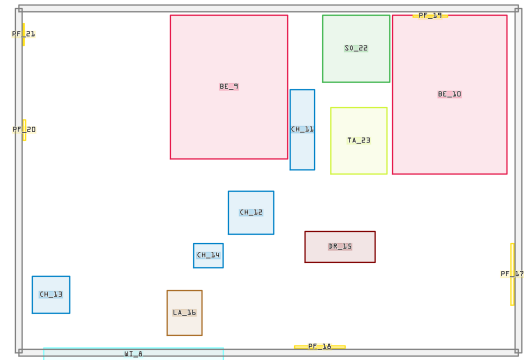
81056: Largest area (265.56 m²) with high object density and complex spatial arrangement.



84230: Medium area (142.69 m²), moderately dense with a balanced spatial distribution.



84052: Smaller area (69.90 m²), moderately cluttered with scattered objects.



83692: Smallest area (40.40 m²), few objects, very sparse layout.

Figure 3.1: Sorted floorplans used in experiments. The images are arranged from the most complex and cluttered (top left) to the simplest and sparsest (bottom right). Each floorplan is rendered from the ASE dataset, illustrating the object footprints and their semantic labels. The complexity is determined based on area size, object density, and spatial arrangement.

```
{
  "floorplan_dimensions": {
    "min_x": -9.03,
    "min_y": -7.20,
    "max_x": -2.33,
    "max_y": 3.21,
    "width": 6.71,
    "height": 10.42
  },
  "obb_metadata": [
    {
      "footprint": [[-8.57, 0.56], [-8.24, 0.56], ...],
      "center_2d": [-8.41, 1.88],
      "sem_id": 28,
      "name": "window",
      "instance": 10,
      "walkable": false,
      "bypassable": false
    },
    ...
  ]
}
```

Figure 3.2: Example snippet of structured OBB metadata in JSON format

```
messages = [
  {
    "role": "system",
    "content": (
      "You are an indoor navigation agent."
      "Given a top-down floorplan image with labelled objects (2-letter + ID) and gray walls,"
      "find a collision-free path between the specified start and goal objects."
      "Output exactly a JSON object {'path': [[x,y], ...]} or {'path': []} if none."
    )
  },
  {
    "role": "user",
    "content": [
      {
        "type": "text",
        "text": (
          f"Compute path from '{src_label}' to '{tgt_label}'."
        )
      },
      {
        "type": "image_url",
        "image_url": {
          "url": f"data:image/png;base64,{base64_image}",
          "detail": "high"
        }
      }
    ]
  }
]
```

Listing 3.1: Floorplan-based Prompt Structure for GPT-4o

Figure 3.3: The prompt contains a system message defining the navigation task and a user message with the source and target object labels along with a base64-encoded floorplan image.

```
# System prompt
system_prompt = {
    "system_message": (
        "You are a navigation AI operating in a 2D indoor environment."
        "You have access to the environment's floorplan metadata in JSON format"
        "with Oriented Bounding Boxes (OBBs)."
```

Listing 3.2: Metadata-based Prompt Structure for GPT-4o

Figure 3.4: The prompt specifies navigation tasks using object names, instances, and coordinates taken from the OBB metadata.

Chapter 4

Critical Evaluation

This chapter shows the evaluation of results related to GPT-4o’s spatial reasoning capabilities in indoor navigation tasks. The experiments compared two distinct modalities: visual inputs from floorplan images and structured inputs obtained from Oriented Bounding Box (OBB) metadata. This evaluation investigates the impact of these input modalities on performance and compares the effectiveness of zero-shot and few-shot prompting methods. The results are also benchmarked against a baseline established by the classical A* path-planning algorithm.

4.1 Metrics

The following metrics were applied to quantitatively assess the generated paths, providing detailed insights into different performance aspects.

- **Incompletion Rate:** Percentage of non-empty paths whose JSON output is incomplete, either due to an unfinished list or an improperly formatted output, indicating that the generation process stopped prematurely.
- **Out-of-Map Error Rate:** Percentage of waypoints positioned outside the defined spatial boundaries.
- **Collision Rate:** Percentage of waypoints colliding with obstacles.
- **Path Validity:** Binary indicator of successful path completion to the goal without obstacle intersection.
- **Path Length Ratio:** Ratio of GPT-4o path length to the A* baseline path length.
- **Waypoint Deviation:** Mean Euclidean distance between GPT-4o and A* waypoints.

4.2 Challenges in Visual Spatial Reasoning

GPT-4o had difficulty understanding spatial relationships from visual floorplan images. The model consistently failed to generate complete and spatially accurate paths across different conditions and levels of path and scene complexity, producing waypoints that were either incomplete or located outside the scene boundaries.

4.2.1 Zero-Shot Learning Setting

As shown in Figure 4.1, the zero-shot learning setting consistently produced either out-of-bound or incomplete paths. Scenes with larger areas and higher object density significantly increased the likelihood of route generation failure. Even in smaller and less dense scenes, although the model completed the path, the generated routes deviated significantly from realistic navigable paths and extended beyond the map boundaries.

This problem arises from the absence of a clearly defined coordinate system. Without clearly defined axes, an origin, a grid, or a numeric scale, GPT-4o is forced to infer spatial relationships directly from the visual input without accurate spatial binding. As a result, the generated coordinates often fall into

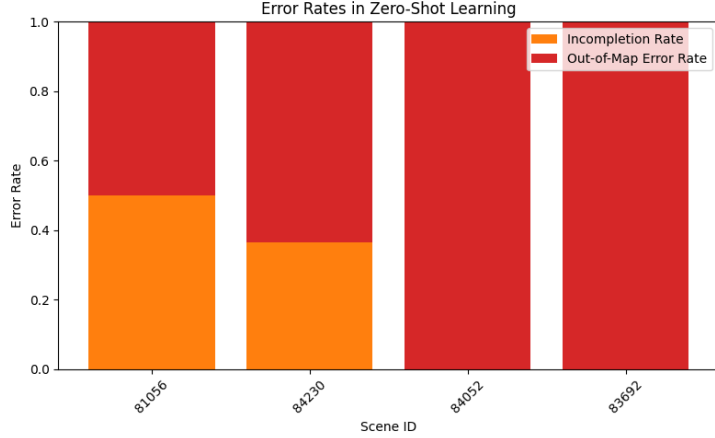


Figure 4.1: Incompletion Rate and Out-of-Map Error Rate observed in the zero-shot learning setting.

unrealistic ranges, from single digits to the hundreds. This shows that the model struggles to translate visual features into accurate path coordinates when no prior examples are given.

4.2.2 Few-Shot Learning Setting

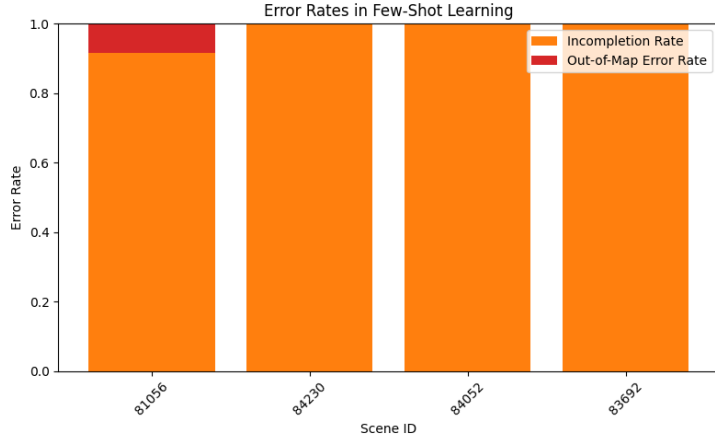


Figure 4.2: Incompletion Rate and Out-of-Map Error Rate observed in the few-shot learning setting.

The few-shot setting did not significantly improve overall performance, as most generated paths remained incomplete, as shown in Figure 4.2. Incorporating several examples with the floorplan image in the prompt can fragment the model’s attention [23]. Since these examples may occupy hundreds of tokens, they reduce the available context length needed for planning longer routes, increasing the likelihood of generating inconsistent or inaccurate paths.

Despite the lack of improvement in overall performance, the few-shot setting significantly improved spatial consistency. Unlike the zero-shot setting, where coordinates spanned an unreasonably wide range, the few-shot setting produced coordinates that were more consistent and closely matched the real coordinate system of the floorplan, typically ranging from about -10 to 10. However, this improvement did not result in the generation of complete and valid paths. Although the few-shot setting made the coordinates appear more realistic by replicating numerical patterns, it did not help the model internalise the fundamental spatial logic. As a result, the model produced paths that seemed consistent but remained invalid.

4.2.3 Limitations in Visual Path Planning

Further analysis showed that the generated path shapes did not align with the spatial positions of the start and goal objects, resulting in routes that deviated from realistic navigable paths. One of the key

causes of the failure may be visual clutter and high object density. Each scene contains numerous coloured bounding boxes, which can make it challenging for GPT-4o to accurately interpret spatial relationships. Furthermore, previous research indicates that Vision Language Models (VLMs) and transformer-based models, including GPT-4o, can sometimes produce hallucinations and biases [14], potentially leading to errors during path generation.

The consistent failures in visual path planning show inherent limitations in GPT-4o’s ability to handle visual perception, particularly when precise geometric reasoning is required. The model struggles to identify navigable paths through obstacles in visually cluttered and ambiguous scenes, showing the difficulties of relying solely on visual representations for geometric tasks.

4.3 Strengths of Structured Spatial Representation

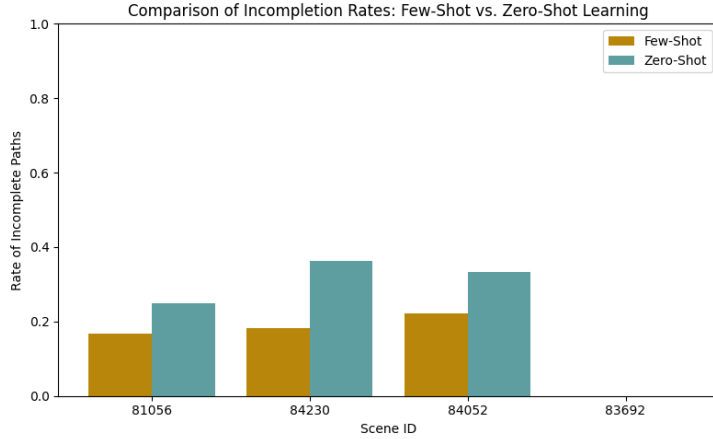


Figure 4.3: Comparison of Incompletion Rates: Few-Shot vs. Zero-Shot Learning across Scenes.

In contrast to floorplan-based navigation, structured OBB metadata provided GPT-4o with clear and unambiguous geometric information. This explicit representation significantly improved the model’s ability to generate complete paths. Compared to floorplan-based navigation, where almost every generated path is incomplete or located outside the scene boundaries, metadata-based navigation significantly reduced the number of incomplete paths in both zero-shot and few-shot learning settings, as shown in Figure 4.3. This shows that GPT-4o performs better when geometric information is explicitly defined since it can map waypoints more accurately and maintain spatial consistency. The structured input helps the model bypass visual interpretation challenges and use precise spatial coordinates and object boundaries directly. In addition, in the few-shot setting, the number of incomplete paths was nearly half of that in the zero-shot setting, indicating better adaptation when given prior examples.

4.4 Inconsistent Performance of Few-Shot Learning

The zero-shot setting produced a higher proportion of valid paths compared to the few-shot setting, in contrast to the expectation that few-shot learning, which includes examples of valid A* paths, would help GPT-4o generate more accurate paths. However, this pattern was not consistent across all scenes. For instance, in Scene 84230, few-shot learning resulted in a higher valid path rate, while in Scenes 84230 and 83692, it produced fewer valid paths, as shown in Figure 4.5.

One possible reason for this inconsistency is that the model becomes too focused on the specific paths presented during training, making it less adaptable to variations. Additionally, the way A* examples are formatted might unintentionally introduce bias. Since the A* planner uses a 0.1 m grid size, the examples may be overly detailed or specific. Consequently, GPT-4o may generate paths that resemble the examples in structure but lack spatial accuracy, as discussed in Subsection 4.2.2.

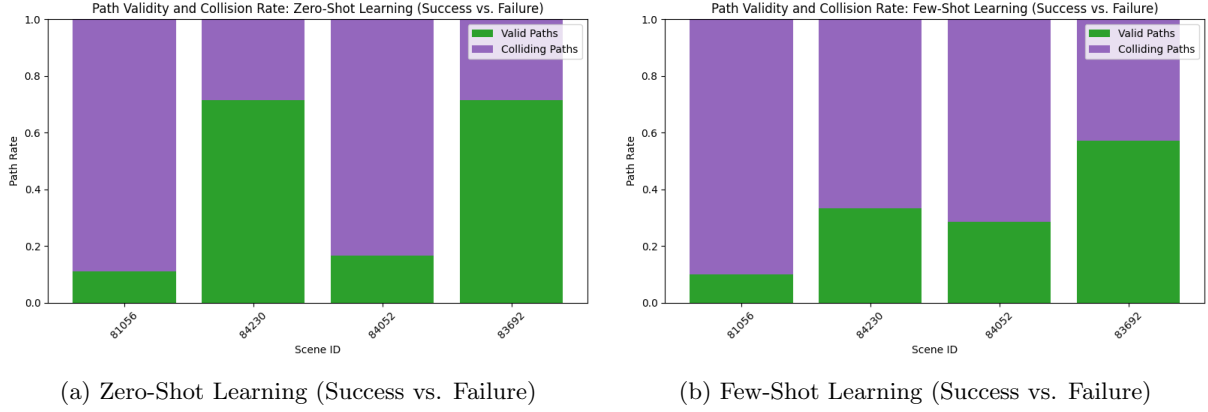


Figure 4.4: Comparison of Path Validity and Collision Rates between Zero-Shot and Few-Shot Learning across Scenes.

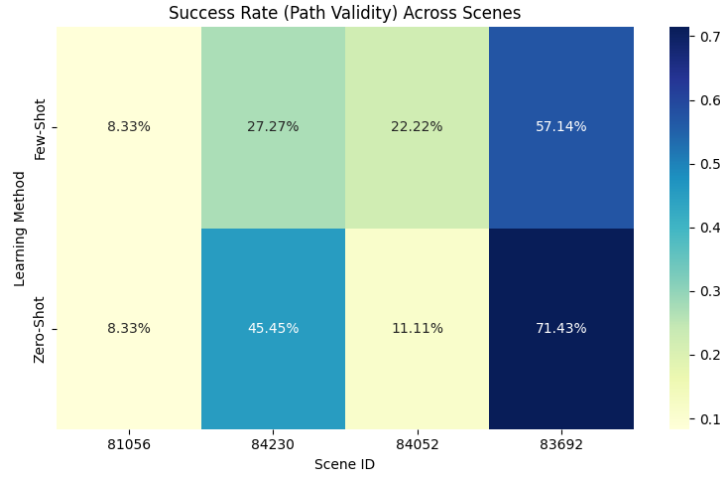


Figure 4.5: Comparison of Success Rates (Path Validity) Across Scenes between Few-Shot and Zero-Shot Learning.

4.5 Comparative Analysis with Classical A*

The classical A* algorithm, used as a geometric baseline, consistently generated near-optimal paths in all scenarios. However, in this project, the A* implementation prioritises wider paths, even if they are slightly longer, to reflect human preferences for safer, more spacious routes rather than risky, narrow ones. As a result, it does not always generate the shortest path. This approach was also incorporated into GPT-4o’s prompts by explicitly instructing the model to avoid narrow gaps.

For the comparative analysis, only structured input was considered, as every path generated by GPT-4o using floorplan-based navigation was either incomplete or out of bounds.

In order to conduct a precise evaluation, each path divided into two groups: valid paths, which do not collide with any obstacles, and colliding paths, where generated waypoints intersect with obstacles. The analysis was conducted separately for zero-shot and few-shot learning settings.

Path Length Ratio: GPT-4o vs. A*

The ratio of GPT-4o path length to the A* path length was calculated to evaluate the relative efficiency of GPT-4o-generated paths compared to the geometric baseline.

Zero-Shot Learning Setting For valid paths, the median path length ratio is slightly greater than 1.0, indicating that GPT-4o generally produces paths comparable in length to A*, with a minor tendency to

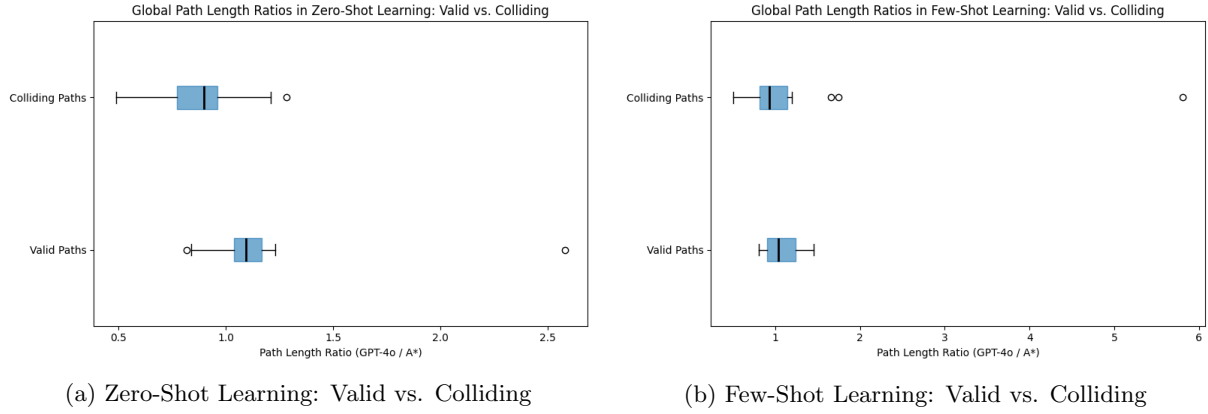


Figure 4.6: Comparison of Global Path Length Ratios between Few-Shot and Zero-Shot Learning: Valid vs. Colliding Paths.

be longer. The box plot for valid paths exhibits a narrow interquartile range (IQR), indicating consistent path lengths close to the A* benchmark.

Although the median path length ratio for colliding paths is close to 1.0, the IQR is more concentrated below 1.0. This indicates that most colliding paths are shorter than the corresponding A* paths. In contrast to valid paths, the colliding paths display more variability, with a few exhibiting deviations.

Few-Shot Learning Setting Compared to the zero-shot setting, both valid and colliding paths in the few-shot setting have a median path length ratio closer to 1.0, with the box plot being more tightly concentrated around this value. This indicates that the valid paths generated by GPT-4o are generally similar in length to those of A*. Additionally, the shorter whiskers indicate reduced variability, and the outliers observed in valid paths are no longer present. This outcome suggests that the model learns to approximate the correct path length from in-context examples, although it still faces challenges with collision avoidance.

In both zero-shot and few-shot learning settings, the median length of colliding paths remains consistently smaller than 1.0. This is because colliding paths generated by GPT-4o frequently pass through obstacles to reach the goal and leads to shorter path lengths, as shown in Appendix C. This is different from A*, which actively avoids obstacles.

Waypoint Deviation: GPT-4o vs. A*

The mean Euclidean distance between waypoints generated by GPT-4o and those produced by the A* algorithm was calculated to evaluate how closely GPT-4o's path planning aligns with the geometric baseline. This analysis identifies differences in spatial accuracy between the two approaches.

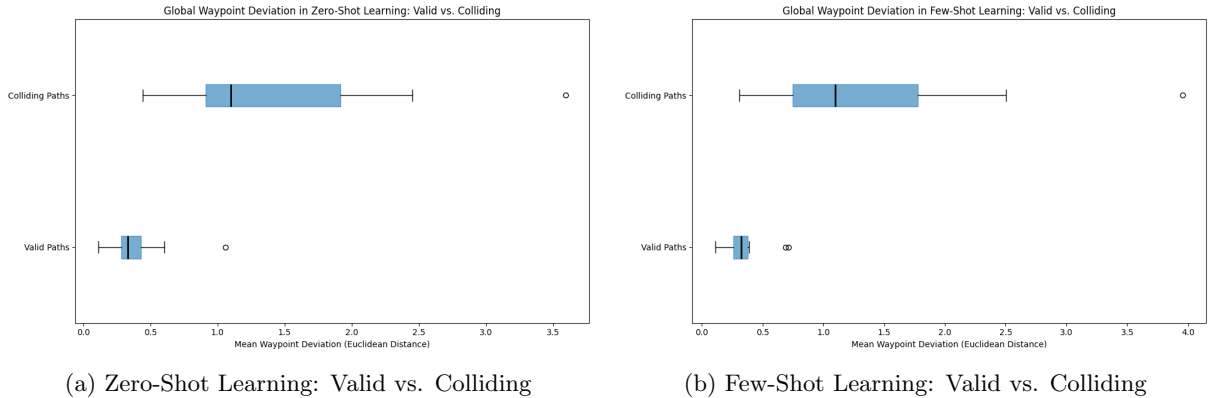


Figure 4.7: Comparison of Global Waypoint Deviations between Few-Shot and Zero-Shot Learning: Valid vs. Colliding Paths.

Zero-Shot Learning Setting For valid paths, the box plot and median are closer to zero, indicating that the mean deviation is relatively small. The narrow shape of the box plot shows low variance, implying consistent waypoint accuracy for valid paths.

In contrast, the box plot for colliding paths is positioned further from zero, reflecting a higher average deviation. The box and whiskers are wider than those for valid paths, indicating greater variability in how much waypoints deviate.

Outliers are present in both valid and colliding paths. In valid paths, the outlier appears just above the upper whisker, indicating a minor deviation from the typical range. In contrast, the outlier in colliding paths is much farther away from the box, reflecting a significantly greater deviation compared to the rest of the data.

Few-Shot Learning Setting Similar to the zero-shot learning setting described earlier, the pattern of waypoint deviation in the few-shot learning condition follows a comparable trend.

A key difference in the few-shot setting is that the box plots for both valid and colliding paths are narrower than in the zero-shot setting. The shorter upper whisker in valid paths indicates that the highest observed deviations are more constrained. This shows that few-shot learning decreases variance in valid path deviations, resulting in more consistent performance across both valid and colliding paths.

Few-shot learning benefits from prior valid paths examples generated by A*, allowing GPT-4o to better understand spatial relationships. These examples act as guidance, prompting the model to generate more consistent and stable outputs. As a result, it reduces the possibility of the model generating waypoints that deviate significantly from the baseline pattern.

GPT-4o’s performance with structured inputs closely matches the A* baseline, demonstrating strong potential for spatial reasoning when geometric conditions are clearly defined. However, in the visual input setting, the model struggled to generate feasible paths, exposing major limitations in its ability to interpret and reason spatially using visual information.

Chapter 5

Conclusion

5.1 Contribution Summary

This project provides an empirical evaluation of GPT-4o’s spatial reasoning abilities in indoor navigation by comparing its performance against classical geometric algorithms such as A*, across different types of input and prompting strategies. A reliable baseline was established using classical A* planning on detailed Oriented Bounding Box (OBB) maps to serve as a benchmark for assessing the planning capabilities of Large Language Models (LLMs). The study investigates GPT-4o’s ability to perform coordinate-based spatial reasoning by using both visual floorplan images and structured OBB metadata. It also compares zero-shot and few-shot prompting to see how in-context examples influence GPT-4o’s spatial reasoning accuracy.

5.2 Future Work

5.2.1 Augmenting Floorplan Inputs with Spatial Context Annotations

As GPT-4o struggles to interpret spatial relationships from visual floorplans inputs, future work could focus on augmenting visual inputs with spatial context annotations. Instead of depending solely on raw object layouts, floorplans could include brief textual descriptions that highlight the spatial relationships and contextual clues obtained from visual data. This multimodal approach would help the model better interpret spatial context and object positioning while avoiding excessive visual clutter. This could reduce reliance on purely geometric inference.

5.2.2 Incorporating Semantic Cues in Few-Shot Prompting

Currently, waypoints are represented only as numerical coordinates, which causes GPT-4o to simply replicate surface patterns. Providing semantic descriptions such as justifications for path selection or indications of movement type could help GPT-4o overcome its existing spatial reasoning limitations. These semantic cues could help the model better understand the rationale behind waypoint placement by utilising GPT-4o’s natural language reasoning abilities.

5.2.3 Hybrid Planning Approaches

A hybrid navigation system can improve pathfinding efficiency and reliability by combining GPT-4o’s semantic reasoning with traditional geometric algorithms like A*. This method uses a language model to create high-level path descriptions and semantic waypoints from contextual prompts, while a geometric planner calculates an optimal path that avoids obstacles. The accurate waypoints are then refined using semantic feedback. By decomposing the system into semantic reasoning and geometric models, both symbolic reasoning and precise geometric planning can be fully utilised.

5.2.4 Self-Corrective Path Planning

This approach was implemented, but due to the cost and format of the feedback mechanism, it showed only minor improvements for simpler paths and had limited effects on more complex cases. The feedback

consisted of the coordinates of failed paths along with collision information. However, the revised paths often followed the original ones without exploring alternative routes.

Although this approach encountered some limitations, it showed potential for further investigation and improvement. For instance, instead of simply providing textual feedback, penalising waypoints with collisions and rewarding successful navigation could encourage the model to adopt corrective behaviours. Additionally, enhancing the feedback format with structured natural language explanations, rather than just listing coordinates, could better utilise GPT-4o’s reasoning capabilities.

Bibliography

- [1] Silya Achat, Julien Marzat, and Julien Moras. Path Planning Incorporating Semantic Information for Autonomous Robot Navigation. In *19th International Conference on Informatics in Control, Automation and Robotics (ICINCO) 2022*, pages 285–295, Lisbonne, Portugal, July 2022. SCITEPRESS - Science and Technology Publications. URL: <https://hal.science/hal-03831940>, doi:10.5220/0011134300003271.
- [2] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Daniel Ho, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Eric Jang, Rosario Jauregui Ruano, Kyle Jeffrey, Sally Jesmonth, Nikhil J Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Kuang-Huei Lee, Sergey Levine, Yao Lu, Linda Luu, Carolina Parada, Peter Pastor, Jornell Quiambao, Kanishka Rao, Jarek Rettinghouse, Diego Reyes, Pierre Sermanet, Nicolas Sievers, Clayton Tan, Alexander Toshev, Vincent Vanhoucke, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Mengyuan Yan, and Andy Zeng. Do as i can, not as i say: Grounding language in robotic affordances, 2022. URL: <https://arxiv.org/abs/2204.01691>, arXiv:2204.01691.
- [3] Peter Anderson, Qi Wu, Damien Teney, Jake Bruce, Mark Johnson, Niko Sünderhauf, Ian Reid, Stephen Gould, and Anton van den Hengel. Vision-and-language navigation: Interpreting visually-grounded navigation instructions in real environments, 2018. URL: <https://arxiv.org/abs/1711.07280>, arXiv:1711.07280.
- [4] Sean L. Bowman, Nikolay Atanasov, Kostas Daniilidis, and George J. Pappas. Probabilistic data association for semantic slam. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1722–1729, 2017. doi:10.1109/ICRA.2017.7989203.
- [5] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners, 2020. URL: <https://arxiv.org/abs/2005.14165>, arXiv:2005.14165.
- [6] Cesar Cadena, Luca Carlone, Henry Carrillo, Yasir Latif, Davide Scaramuzza, José Neira, Ian Reid, and John J. Leonard. Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age. *IEEE Transactions on Robotics*, 32(6):1309–1332, 2016. doi:10.1109/TR0.2016.2624754.
- [7] Angel Chang, Angela Dai, Thomas Funkhouser, Maciej Halber, Matthias Niessner, Manolis Savva, Shuran Song, Andy Zeng, and Yinda Zhang. Matterport3d: Learning from rgb-d data in indoor environments. *International Conference on 3D Vision (3DV)*, 2017.
- [8] Devendra Singh Chaplot, Dhiraj Gandhi, Saurabh Gupta, Abhinav Gupta, and Ruslan Salakhutdinov. Learning to explore using active neural slam, 2020. URL: <https://arxiv.org/abs/2004.05155>, arXiv:2004.05155.
- [9] Devendra Singh Chaplot, Dhiraj Prakashchand Gandhi, Abhinav Gupta, and Russ R Salakhutdinov. Object goal navigation using goal-oriented semantic exploration. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 4247–4258. Curran Associates, Inc., 2020. URL: https://proceedings.neurips.cc/paper_files/paper/2020/file/2c75cf2681788adaca63aa95ae028b22-Paper.pdf.

- [10] Jiaoyan Chen, Yuxia Geng, Zhuo Chen, Ian Horrocks, Jeff Z. Pan, and Huajun Chen. Knowledge-aware zero-shot learning: Survey and perspective, 2021. URL: <https://arxiv.org/abs/2103.00070>, [arXiv:2103.00070](#).
- [11] Shizhe Chen, Pierre-Louis Guhur, Cordelia Schmid, and Ivan Laptev. History aware multimodal transformer for vision-and-language navigation, 2023. URL: <https://arxiv.org/abs/2110.13309>, [arXiv:2110.13309](#).
- [12] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, Albert Webson, Shixiang Shane Gu, Zhuyun Dai, Mirac Suzgun, Xinyun Chen, Aakanksha Chowdhery, Alex Castro-Ros, Marie Pellat, Kevin Robinson, Dasha Valter, Sharan Narang, Gaurav Mishra, Adams Yu, Vincent Zhao, Yanping Huang, Andrew Dai, Hongkun Yu, Slav Petrov, Ed H. Chi, Jeff Dean, Jacob Devlin, Adam Roberts, Denny Zhou, Quoc V. Le, and Jason Wei. Scaling instruction-finetuned language models, 2022. URL: <https://arxiv.org/abs/2210.11416>, [arXiv:2210.11416](#).
- [13] Jonathan Crespo, Jose Carlos Castillo, Oscar Martinez Mozos, and Ramon Barber. Semantic information for robot navigation: A survey. *Applied Sciences*, 10(2), 2020. URL: <https://www.mdpi.com/2076-3417/10/2/497>, [doi:10.3390/app10020497](#).
- [14] David DeFazio, Hrudayangam Mehta, Jeremy Blackburn, and Shiqi Zhang. Vision language models can parse floor plan maps, 2024. URL: <https://arxiv.org/abs/2409.12842>, [arXiv:2409.12842](#).
- [15] Venkata Naren Devarakonda, Raktim Gautam Goswami, Ali Umut Kaypak, Naman Patel, Rooholla Khorrambakht, Prashanth Krishnamurthy, and Farshad Khorrami. Orionnav: Online planning for robot autonomy with context-aware llm and open-vocabulary semantic scene graphs, 2024. URL: <https://arxiv.org/abs/2410.06239>, [arXiv:2410.06239](#).
- [16] Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, Wenlong Huang, Yevgen Chebotar, Pierre Sermanet, Daniel Duckworth, Sergey Levine, Vincent Vanhoucke, Karol Hausman, Marc Toussaint, Klaus Greff, Andy Zeng, Igor Mordatch, and Pete Florence. Palm-e: An embodied multimodal language model, 2023. URL: <https://arxiv.org/abs/2303.03378>, [arXiv:2303.03378](#).
- [17] Dyke Ferber, Georg Wölflein, Isabella C. Wiest, Marta Ligeró, Srividhya Sainath, Narmin Ghafari Laleh, Omar S. M. El Nahhas, Gustav Müller-Franzes, Dirk Jäger, Daniel Truhn, and Jakob Nikolas Kather. In-context learning enables multimodal large language models to classify cancer pathology images. *Nature Communications*, 15(1):10104, 2024. [doi:10.1038/s41467-024-51465-9](#).
- [18] David Ferguson, Maxim Likhachev, and Anthony (Tony) Stentz. A guide to heuristic-based path planning. In *Proceedings of ICAPS '05 Workshop on Planning under Uncertainty for Autonomous Systems*, June 2005.
- [19] Daniel Foead, Alifio Ghifari, Marchel Kusuma, Novita Hanafiah, and Eric Gunawan. A systematic literature review of a* pathfinding. *Procedia Computer Science*, 179:507–514, 01 2021. [doi:10.1016/j.procs.2021.01.034](#).
- [20] Daniel Fried, Ronghang Hu, Volkan Cirik, Anna Rohrbach, Jacob Andreas, Louis-Philippe Morency, Taylor Berg-Kirkpatrick, Kate Saenko, Dan Klein, and Trevor Darrell. Speaker-follower models for vision-and-language navigation, 2018. URL: <https://arxiv.org/abs/1806.02724>, [arXiv:1806.02724](#).
- [21] Daniel Fried, Ronghang Hu, Volkan Cirik, Anna Rohrbach, Jacob Andreas, Louis-Philippe Morency, Taylor Berg-Kirkpatrick, Kate Saenko, Dan Klein, and Trevor Darrell. Speaker-follower models for vision-and-language navigation, 2018. URL: <https://arxiv.org/abs/1806.02724>, [arXiv:1806.02724](#).
- [22] C. Galindo, A. Saffiotti, S. Coradeschi, P. Buschka, J.A. Fernandez-Madrigal, and J. Gonzalez. Multi-hierarchical semantic maps for mobile robotics. In *2005 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 2278–2283, 2005. [doi:10.1109/IR0S.2005.1545511](#).
- [23] Alberto Gandolfi. Gpt-4 in education: Evaluating aptness, reliability, and loss of coherence in solving calculus problems and grading submissions. *International Journal of Artificial Intelligence in Education*, 35(1):367–397, March 2025. [doi:10.1007/s40593-024-00403-3](#).

- [24] Muhammad Fadhil Ginting, Sung-Kyun Kim, David D. Fan, Matteo Palieri, Mykel J. Kochenderfer, and Ali akbar Agha-mohammadi. SEEK: Semantic reasoning for object goal navigation in real world inspection tasks. In *Proc. of Robotics: Science and Systems*, 2024.
- [25] S. Gottschalk, M. C. Lin, and D. Manocha. *OBbTree: A Hierarchical Structure for Rapid Interference Detection*. Association for Computing Machinery, New York, NY, USA, 1 edition, 2023. URL: <https://doi.org/10.1145/3596711.3596791>.
- [26] Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A. Smith. Don’t stop pretraining: Adapt language models to domains and tasks, 2020. URL: <https://arxiv.org/abs/2004.10964>, [arXiv:2004.10964](https://arxiv.org/abs/2004.10964).
- [27] Tianbao Hao, Yongshan Liu, Xiang Gong, Dehan Kong, and Jianjun Wang. Visibility detection of 3d objects and visual k-nearest neighbor query based on convex hull model. *Mathematical Problems in Engineering*, 2022:1–14, 06 2022. doi:10.1155/2022/8302974.
- [28] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968. doi:10.1109/TSSC.1968.300136.
- [29] Nathan Hughes, Yun Chang, Siyi Hu, Rajat Talak, Rumaisa Abdulhai, Jared Strader, and Luca Carlone. Foundations of spatial perception for robotics: Hierarchical representations and real-time systems, 2023. URL: <https://arxiv.org/abs/2305.07154>, [arXiv:2305.07154](https://arxiv.org/abs/2305.07154).
- [30] Karthik Karur, Nitin Sharma, Chinmay Dharmatti, and Joshua E. Siegel. A survey of path planning algorithms for mobile robots. *Vehicles*, 3(3):448–468, 2021. URL: <https://www.mdpi.com/2624-8921/3/3/27>, doi:10.3390/vehicles3030027.
- [31] Ioannis Kostavelis and Antonios Gasteratos. Semantic mapping for mobile robotics tasks: A survey. *Robotics and Autonomous Systems*, 66:86–103, 2015. URL: <https://www.sciencedirect.com/science/article/pii/S0921889014003030>, doi:10.1016/j.robot.2014.12.006.
- [32] Labelbox. Zero-shot learning vs. few-shot learning vs. fine-tuning: A technical walk-through using openai’s apis & models, 2023. URL: <https://labelbox.com/guides/zero-shot-learning-few-shot-learning-fine-tuning/>.
- [33] Steven M. LaValle. Rapidly-exploring random trees : a new tool for path planning. *The annual research report*, 1998. URL: <https://api.semanticscholar.org/CorpusID:14744621>.
- [34] Daniel Lenton, Stephen James, Ronald Clark, and Andrew J. Davison. End-to-end egospheric spatial memory, 2021. URL: <https://arxiv.org/abs/2102.07764>, [arXiv:2102.07764](https://arxiv.org/abs/2102.07764).
- [35] Junjia Liu, Jianfei Guo, Zehui Meng, and Jingtao Xue. Revolt: Relational reasoning and voronoi local graph planning for target-driven navigation, 2023. URL: <https://arxiv.org/abs/2301.02382>, [arXiv:2301.02382](https://arxiv.org/abs/2301.02382).
- [36] Katherine Liu, Martina Stadler, and Nicholas Roy. Learned sampling distributions for efficient planning in hybrid geometric and object-level representations. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pages 9555–9562, 2020. doi:10.1109/ICRA40945.2020.9196771.
- [37] Lixing Liu, Xu Wang, Xin Yang, Hongjie Liu, Jianping Li, and Pengfei Wang. Path planning techniques for mobile robots: Review and prospect. *Expert Systems with Applications*, 227:120254, 2023. URL: <https://www.sciencedirect.com/science/article/pii/S095741742300756X>, doi:10.1016/j.eswa.2023.120254.
- [38] Meta Research. Aria synthetic environments dataset. <https://www.projectaria.com/datasets/ase/>. Project Aria.
- [39] Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, Charles Sutton, and Augustus Odena. Show your work: Scratchpads for intermediate computation with language models, 2021. URL: <https://arxiv.org/abs/2112.00114>, [arXiv:2112.00114](https://arxiv.org/abs/2112.00114).
- [40] OpenAI. Gpt-4o system card, 2024. URL: <https://arxiv.org/abs/2410.21276>, [arXiv:2410.21276](https://arxiv.org/abs/2410.21276).

- [41] Charles R. Qi, Or Litany, Kaiming He, and Leonidas J. Guibas. Deep hough voting for 3d object detection in point clouds, 2019. URL: <https://arxiv.org/abs/1904.09664>, arXiv:1904.09664.
- [42] Hongwei Qin, Shiliang Shao, Ting Wang, Xiaotian Yu, Yi Jiang, and Zonghan Cao. Review of autonomous path planning algorithms for mobile robots. *Drones*, 7(3), 2023. URL: <https://www.mdpi.com/2504-446X/7/3/211>, doi:10.3390/drones7030211.
- [43] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision, 2021. URL: <https://arxiv.org/abs/2103.00020>, arXiv:2103.00020.
- [44] Krishan Rana, Jesse Haviland, Sourav Garg, Jad Abou-Chakra, Ian Reid, and Niko Suenderhauf. Sayplan: Grounding large language models using 3d scene graphs for scalable task planning. In *7th Annual Conference on Robot Learning*, 2023. URL: <https://openreview.net/forum?id=wMpOM00Ss7a>.
- [45] Sonia Raychaudhuri and Angel X. Chang. Semantic mapping in indoor embodied ai – a comprehensive survey and future directions, 2025. URL: <https://arxiv.org/abs/2501.05750>, arXiv:2501.05750.
- [46] Assem Sadek, Guillaume Bono, Boris Chidlovskii, Atilla Baskurt, and Christian Wolf. Multi-object navigation in real environments using hybrid policies, 2024. URL: <https://arxiv.org/abs/2401.13800>, arXiv:2401.13800.
- [47] Manolis Savva, Abhishek Kadian, Oleksandr Maksymets, Yili Zhao, Erik Wijmans, Bhavana Jain, Julian Straub, Jia Liu, Vladlen Koltun, Jitendra Malik, Devi Parikh, and Dhruv Batra. Habitat: A Platform for Embodied AI Research. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.
- [48] Zachary Seymour, Niluthpol Chowdhury Mithun, Han-Pang Chiu, Supun Samarasekera, and Rakesh Kumar. Graphmapper: Efficient visual navigation by scene graph generation, 2022. URL: <https://arxiv.org/abs/2205.08325>, arXiv:2205.08325.
- [49] Sakib Shahriar, Brady Lund, Nishith Reddy Mannuru, Muhammad Arbab Arshad, Kadhim Hayawi, Ravi Varma Kumar Bevara, Aashrith Mannuru, and Laiba Batool. Putting gpt-4o to the sword: A comprehensive evaluation of language, vision, speech, and multimodal proficiency, 2024. URL: <https://arxiv.org/abs/2407.09519>, arXiv:2407.09519.
- [50] Chan Hee Song, Valts Blukis, Jonathan Tremblay, Stephen Tyree, Yu Su, and Stan Birchfield. Robospatial: Teaching spatial understanding to 2d and 3d vision-language models for robotics, 2025. URL: <https://arxiv.org/abs/2411.16537>, arXiv:2411.16537.
- [51] Anthony (Tony) Stentz. The d* algorithm for real-time planning of optimal traverses. Technical Report CMU-RI-TR-94-37, Carnegie Mellon University, Pittsburgh, PA, October 1994.
- [52] Hao Tan, Licheng Yu, and Mohit Bansal. Learning to navigate unseen environments: Back translation with environmental dropout, 2019. URL: <https://arxiv.org/abs/1904.04195>, arXiv:1904.04195.
- [53] Marcus Tantakoun, Xiaodan Zhu, and Christian Muise. Llms as planning modelers: A survey for leveraging large language models to construct automated planning models, 2025. URL: <https://arxiv.org/abs/2503.18971>, arXiv:2503.18971.
- [54] Maria Tsimpoukelli, Jacob Menick, Serkan Cabi, S. M. Ali Eslami, Oriol Vinyals, and Felix Hill. Multimodal few-shot learning with frozen language models, 2021. URL: <https://arxiv.org/abs/2106.13884>, arXiv:2106.13884.
- [55] Shrihari Vasudevan, Stefan Gächter, Viet Nguyen, and Roland Siegwart. Cognitive maps for mobile robots—an object based approach. *Robotics and Autonomous Systems*, 55(5):359–371, 2007. From Sensors to Human Spatial Concepts. URL: <https://www.sciencedirect.com/science/article/pii/S0921889006002004>, doi:10.1016/j.robot.2006.12.008.

- [56] Jason Wei, Maarten Bosma, Vincent Y. Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V. Le. Finetuned language models are zero-shot learners, 2022. URL: <https://arxiv.org/abs/2109.01652>, [arXiv:2109.01652](#).
- [57] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023. URL: <https://arxiv.org/abs/2201.11903>, [arXiv:2201.11903](#).
- [58] Abdelrhman Werby, Chenguang Huang, Martin Büchner, Abhinav Valada, and Wolfram Burgard. Hierarchical open-vocabulary 3d scene graphs for language-grounded robot navigation. In *Robotics: Science and Systems XX*, RSS2024. Robotics: Science and Systems Foundation, July 2024. URL: <http://dx.doi.org/10.15607/RSS.2024.XX.077>, [doi:10.15607/rss.2024.xx.077](#).
- [59] Yongqin Xian, Bernt Schiele, and Zeynep Akata. Zero-shot learning – the good, the bad and the ugly, 2020. URL: <https://arxiv.org/abs/1703.04394>, [arXiv:1703.04394](#).
- [60] Binbin Xu, Allen Tao, Hugues Thomas, Jian Zhang, and Timothy D. Barfoot. Makeway: Object-aware costmaps for proactive indoor navigation using lidar, 2024. URL: <https://arxiv.org/abs/2408.17034>, [arXiv:2408.17034](#).
- [61] Duzhen Zhang, Yahan Yu, Jiahua Dong, Chenxing Li, Dan Su, Chenhui Chu, and Dong Yu. MM-LLMs: Recent advances in MultiModal large language models. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Findings of the Association for Computational Linguistics: ACL 2024*, pages 12401–12430, Bangkok, Thailand, August 2024. Association for Computational Linguistics. URL: <https://aclanthology.org/2024.findings-acl.738/>, [doi:10.18653/v1/2024.findings-acl.738](#).
- [62] Matt Zucker, Nathan Ratliff, Anca D. Dragan, Mihail Pivtoraiko, Matthew Klingensmith, Christopher M. Dellin, J. Andrew Bagnell, and Siddhartha S. Srinivasa. Chomp: Covariant hamiltonian optimization for motion planning. *The International Journal of Robotics Research*, 32(9-10):1164–1193, 2013. [arXiv:https://doi.org/10.1177/0278364913488805](#), [doi:10.1177/0278364913488805](#).

Appendix A

Appendix A: AI Prompts/Tools

TikZ Diagram Generation I used ChatGPT to generate a TikZ figure. Example prompts were: “I want a TikZ diagram which has this shape and this content.”, “I created the figure, but I want to add different colours to the headings.”

Grammar Check I used ChatGPT to check my grammar. An example prompt was: “Check the grammar.”

Writing Tone Check I used ChatGPT to check whether my writing tone is academic. An example prompt was: “Does my writing tone sound academic?”

Fix Figure Overflow Issue I used ChatGPT to fix a figure overflow issue. An example prompt was: “My figure overflows the page. How do I fix it?”

Revise Prompt for Clarity I used ChatGPT to revise the prompt that I used in my experiment, to make it easily interpretable by GPT. An example prompt was: “This is the prompt I will feed into GPT-4o. Is it clear enough for GPT-4o to understand without confusion? If not, can you help me revise it so the model can interpret it effectively?”

Using the OpenAI API I used ChatGPT to troubleshoot calls to the OpenAI API. An example prompt was: “I used the code from the official documentation to call the OpenAI API, but it doesn’t work. Can you help me fix it?”

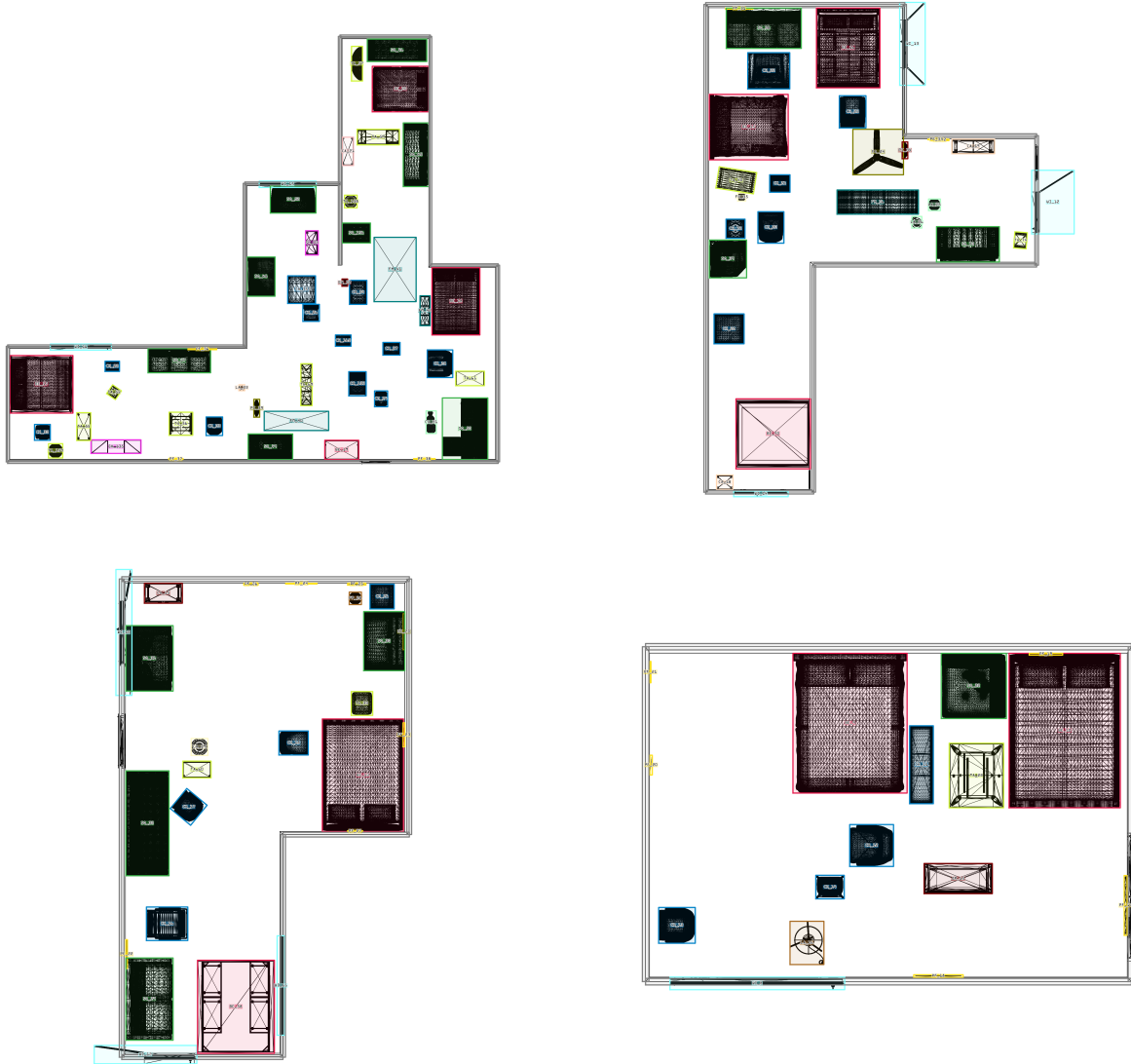
Debugging the Code I used ChatGPT to debug the code. An example prompt was: “I got this error. Why?”

Understanding Concepts I used ChatGPT to clarify concepts I found confusing in research papers. An example prompt was: “Can you explain this in detail?”

Appendix B

Flattened Floorplans with 3D Mesh and OBBs

The following figures are the original 2D floorplans generated by flattening the full 3D mesh along with the projected OBBs. These images show the additional visual complexity introduced by non-essential geometry.



Appendix C

Visual Comparison of GPT-4o and A* Path Planning

The following figures display all complete paths generated by GPT-4o and A*. Corresponding paths are shown in the same colour, with red markers indicating the first collision point with an obstacle.

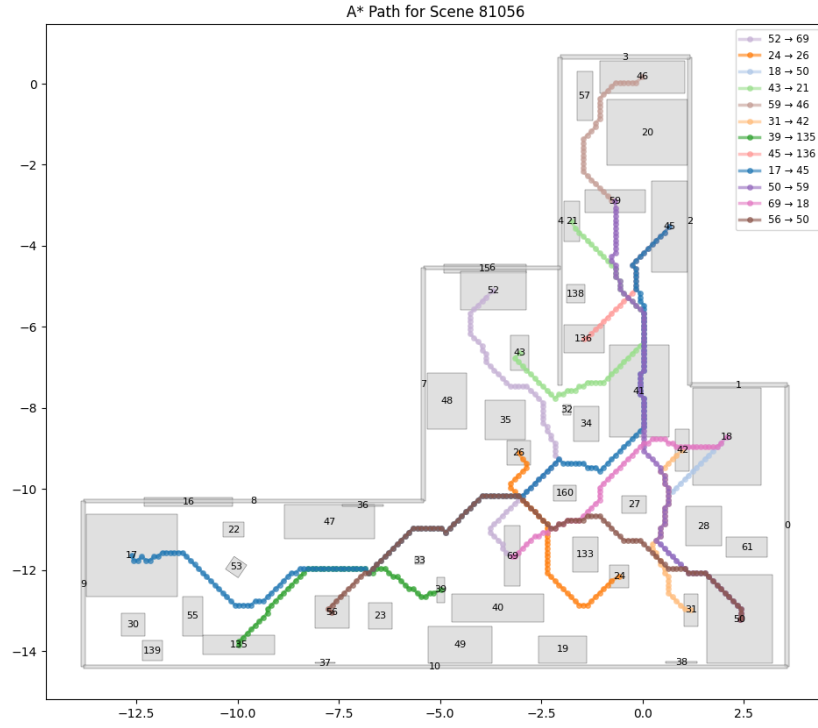


Figure C.1: A* Path for Scene 81056

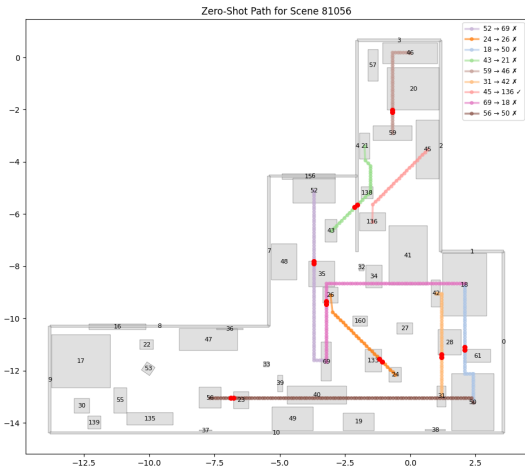


Figure C.2: Zero-Shot Path for Scene 81056

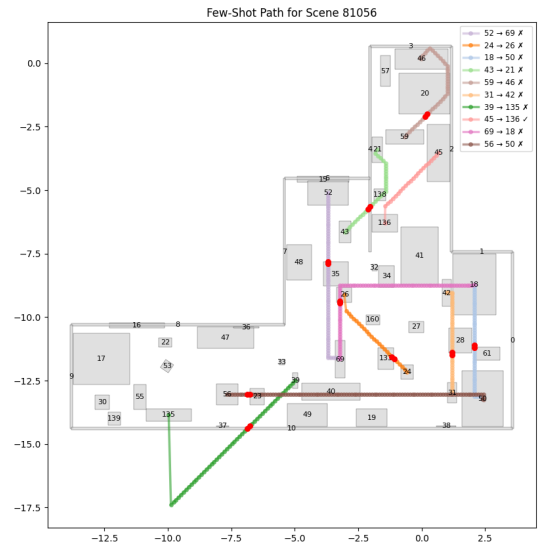


Figure C.3: Few-Shot Path for Scene 81056

Figure C.4: Comparison of A*, Zero-Shot, and Few-Shot Paths for Scene 81056

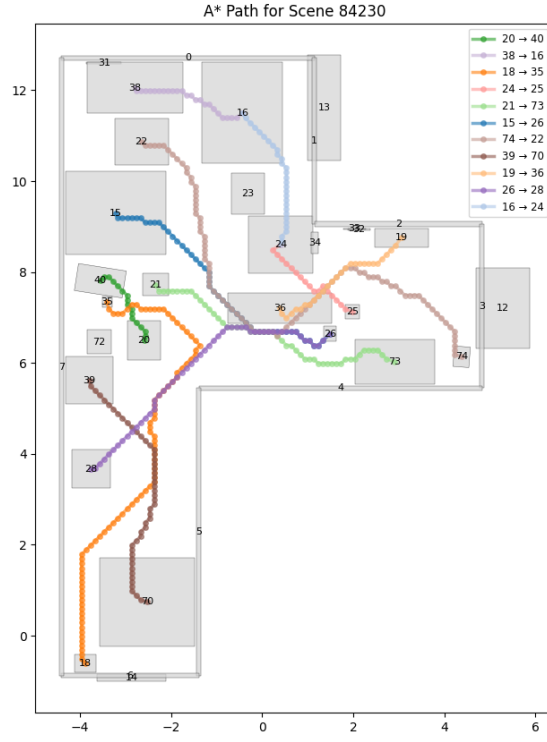


Figure C.5: A* Path for Scene 84230

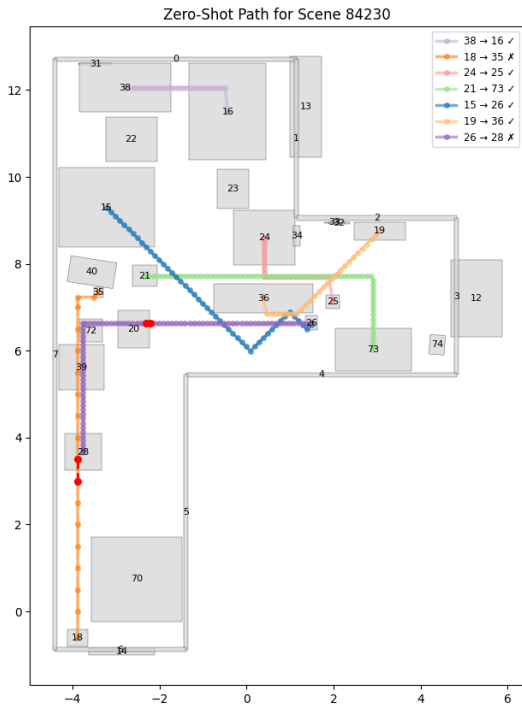


Figure C.6: Zero-Shot Path for Scene 84230

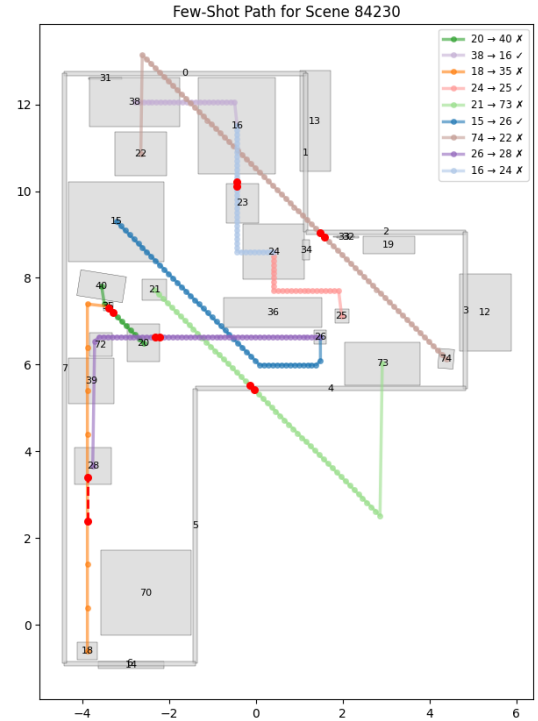


Figure C.7: Few-Shot Path for Scene 84230

Figure C.8: Comparison of A*, Zero-Shot, and Few-Shot Paths for Scene 84230

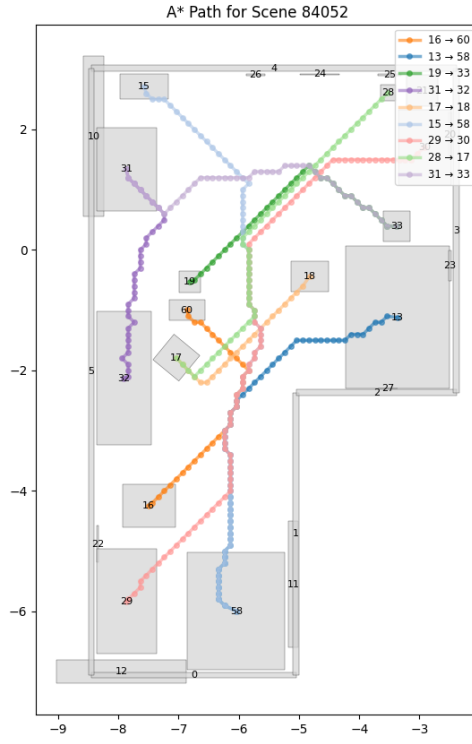


Figure C.9: A* Path for Scene 84052

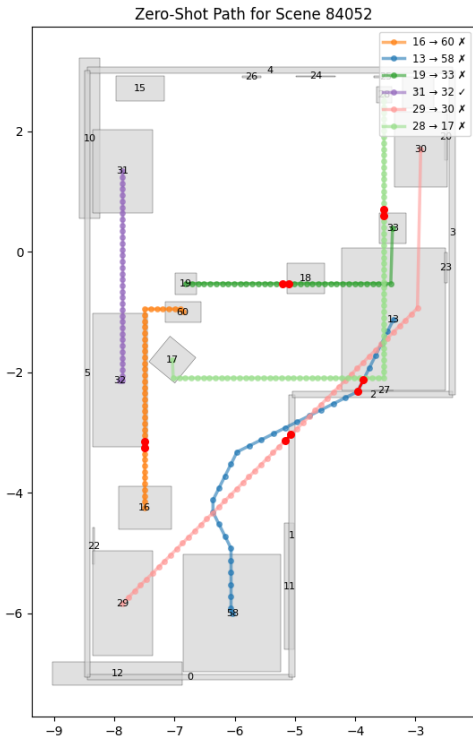


Figure C.10: Zero-Shot Path for Scene 84052

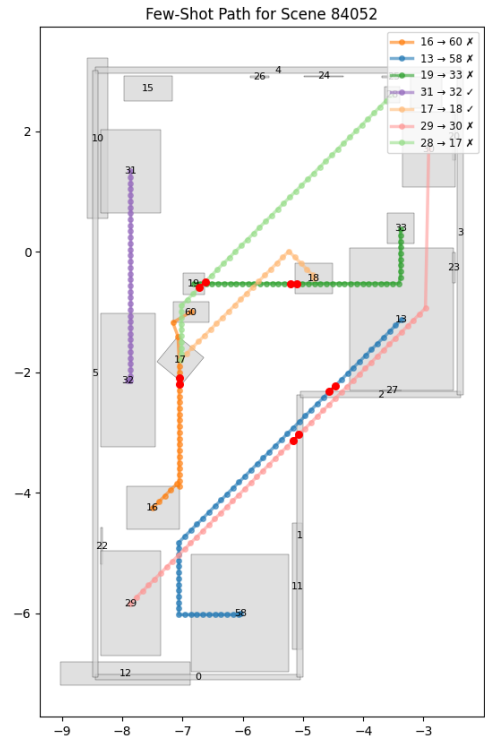


Figure C.11: Few-Shot Path for Scene 84052

Figure C.12: Comparison of A*, Zero-Shot, and Few-Shot Paths for Scene 84052

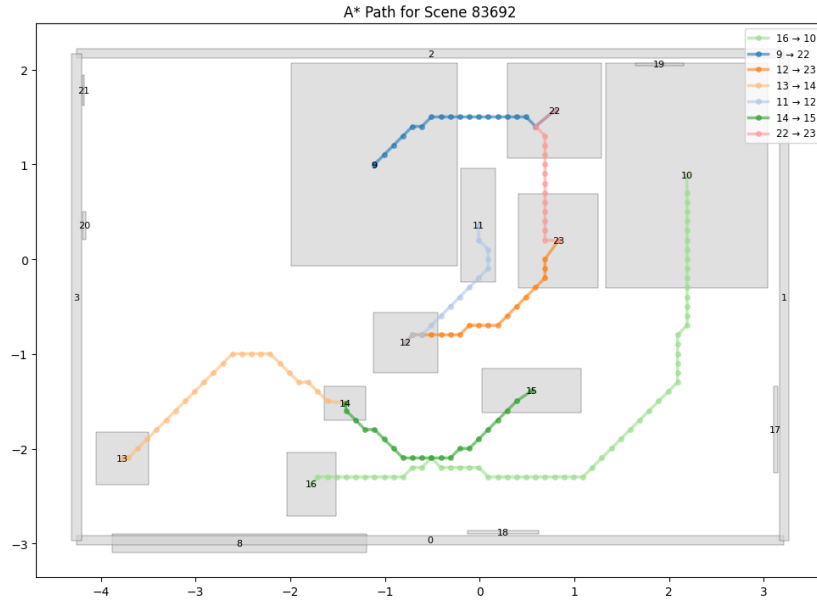


Figure C.13: A* Path for Scene 83692

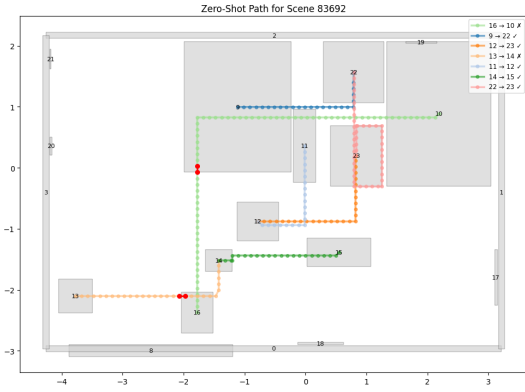


Figure C.14: Zero-Shot Path for Scene 83692

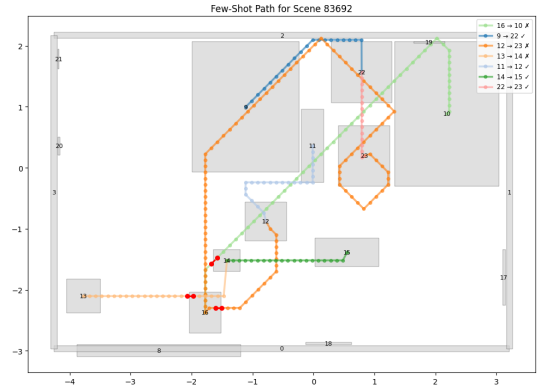


Figure C.15: Few-Shot Path for Scene 83692

Figure C.16: Comparison of A*, Zero-Shot, and Few-Shot Paths for Scene 83692